Report : Advanced Big Data Analytics / Assignment I

Project : Smart Energy Grid Analytics

Ingestion, storage, aggregates, dashboards and day-ahead forecasting for 4,000 smart meters

Group member:

- Mushambo Godfrey 101376
- Umuhoza Scovia 101374
- Manzi Shema Christian 101382

TABLE OF CONTENTS

1.1 Confirm the environment 4

1.2 Create the database structures 7

Data dictionary 8

1.3 Capture and publish a pilot stream 10

2 Capture and prepare the historical dataset 12

2.1 Capture and load the complete stream 12

2.2 Investigate data quality 14

2.3 Clean and preserve a consistent benchmark 15

3 Compare chunk intervals 17

3.1 Create and verify three configurations 17

3.2 Implement and time the five workloads 19

3.3 Explain the chunk results 20

4 Compare compression strategies 22

4.1 Create the two compressed configurations 22

4.2 Measure storage and compression cost 24

4.3 Repeat the query comparison 25

4.4 Select a storage configuration 26

5 Investigate the utility data 27

5.1 Identify demand patterns 27

5.2 Compare regions and customer types 30

5.3 Assess reporting quality 31

5.4 Investigate abnormal consumption 33

5.5 Management recommendation 34

6 Create and investigate continuous aggregates 34

6.1 Create the three aggregate views 34

6.2 Test freshness and delayed arrivals 35

7.1 Build the Raw dashboard 36

7.2 Build the Aggregate dashboard 37

7.3 Verify matching results 38

7.4 Measure performance and judge the tradeoff 39

8 Forecast the next day 40

8.1 Prepare chronological data and features 41

8.2 Evaluate the previous-day baseline 42

8.3 Train and evaluate one regression model 42

8.4 Judge forecast usefulness 43

Document contributions 45

1 Build the ingestion pipeline

1.1 Confirm the environment

PostgreSQL 16.15 with TimescaleDB 2.30.2 (image timescale/timescaledb:latest-pg16) and EMQX run as two containers started with one docker-compose.yml, so both services are monitored in Docker Desktop. The database iot_assignment holds every table and view. TimescaleDB is enabled with CREATE EXTENSION timescaledb. The database time zone is set with ALTER DATABASE iot_assignment SET timezone = 'Africa/Kigali', and daily buckets are aligned to CAT midnight with time_bucket('1 day', event_time, 'Africa/Kigali'). Check: a reading at 23:30 CAT on the 7th falls in the bucket starting 2026-09-07 00:00:00+02, and one just after midnight in the bucket of the 8th (tested; it does not follow UTC midnight, which is 02:00 CAT).

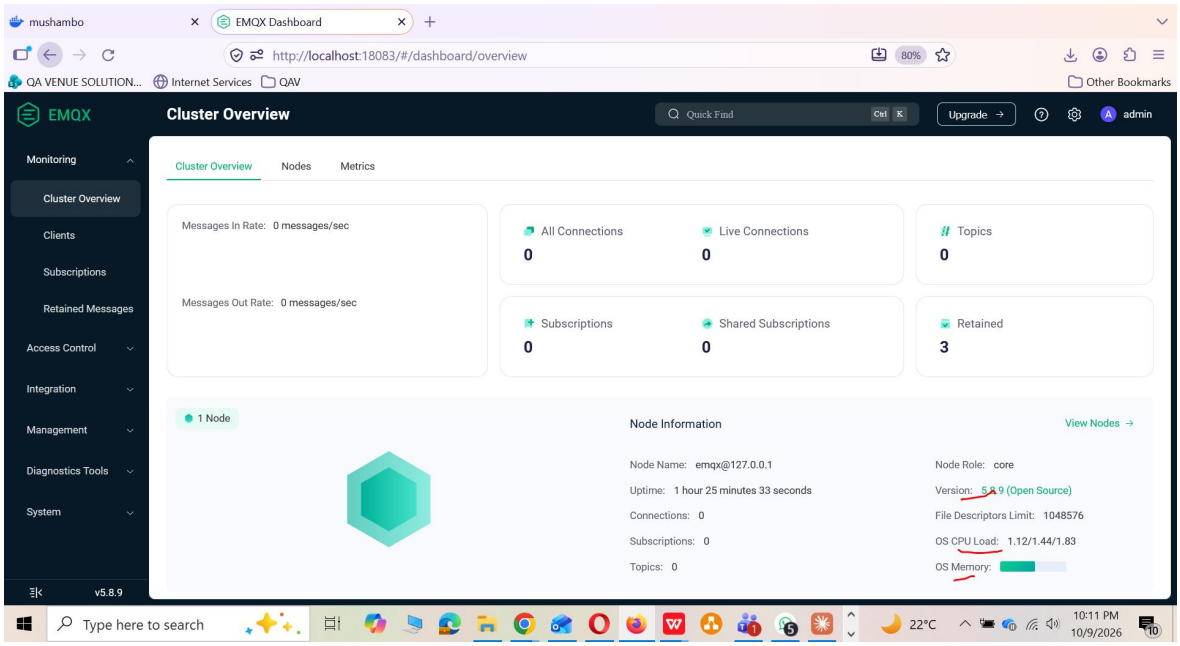


Figure 1. Running EMQX broker

```
PS C:\iot-project> docker compose ps
NAME          IMAGE                                  COMMAND                  SERVICE       CREATED          STATUS          PORTS
emqx          emqx/emqx:latest                       "/usr/bin/docker-ent…"   emqx          55 seconds ago   Up 54 seconds   0.0.0.0:1883->1883/tcp, [::]:1883->1883/tcp, 0.0.0.0:18083->18083/tcp, [::]:18083->18083/tcp
timescaledb   timescale/timescaledb:latest-pg16      "docker-entrypoint.s…"   timescaledb   55 seconds ago   Up 53 seconds   0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp
PS C:\iot-project>
```

Figure 2. `docker compose ps`: both containers up.

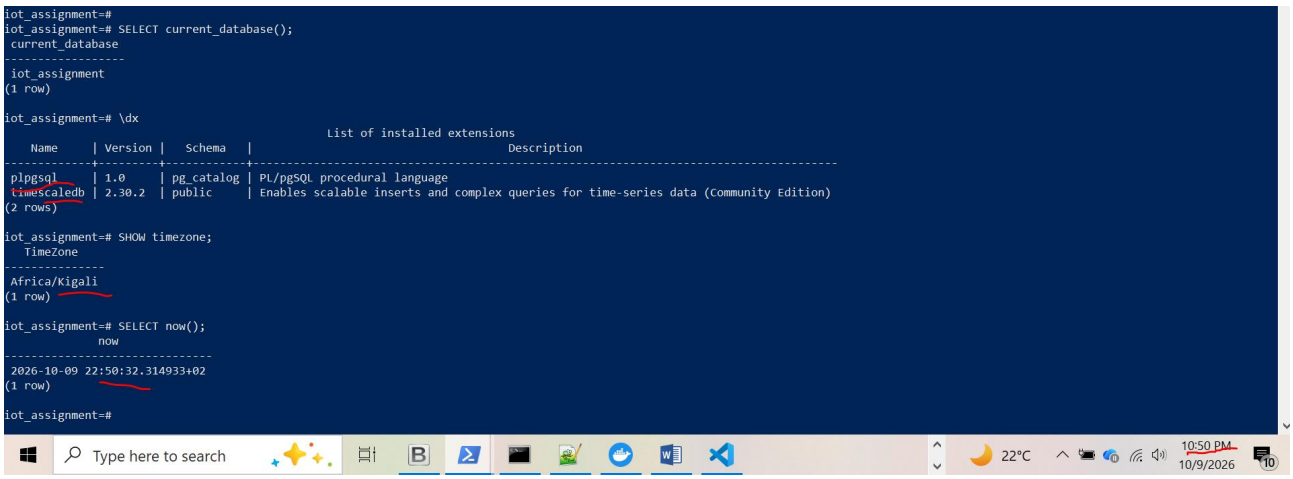


Figure 3. Connection to `iot_assignment`, `\dx` showing the timescaledb extension, and `SHOW timezone` = Africa/Kigali.

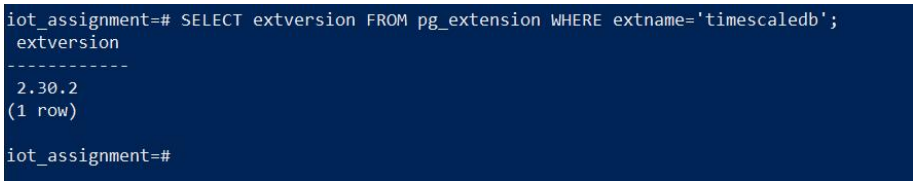


Figure 4. TimescaleDB extension version 2.30.2.

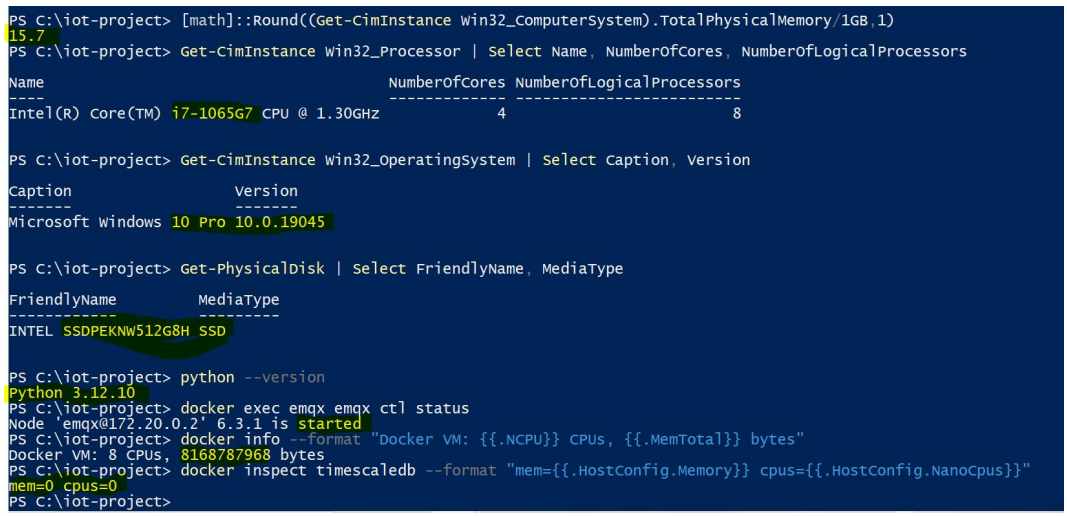


Figure 5. Environment record: CPU, RAM, OS, storage, Python, EMQX status and Docker VM resources.

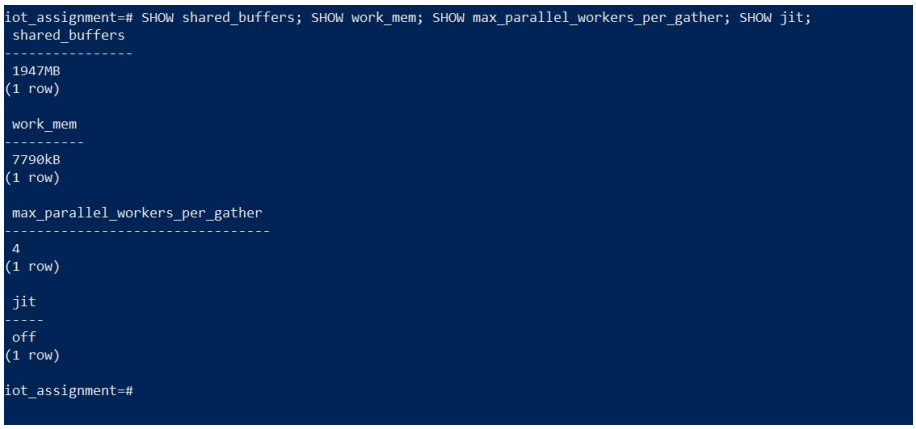


Figure 6. PostgreSQL settings: shared_buffers, work_mem, max_parallel_workers_per_gather, jit .

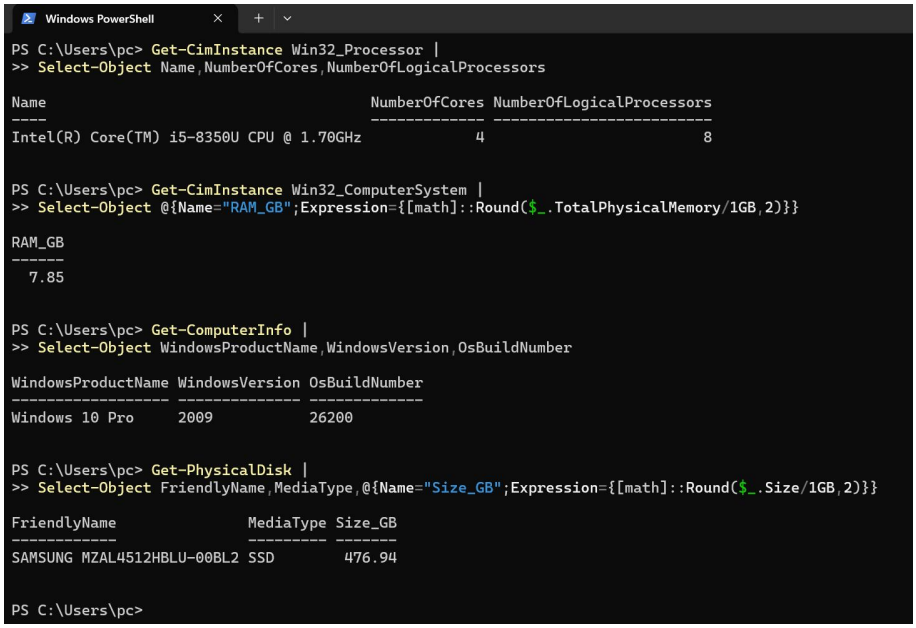


Figure 7. CPU, RAM, OS and storage.

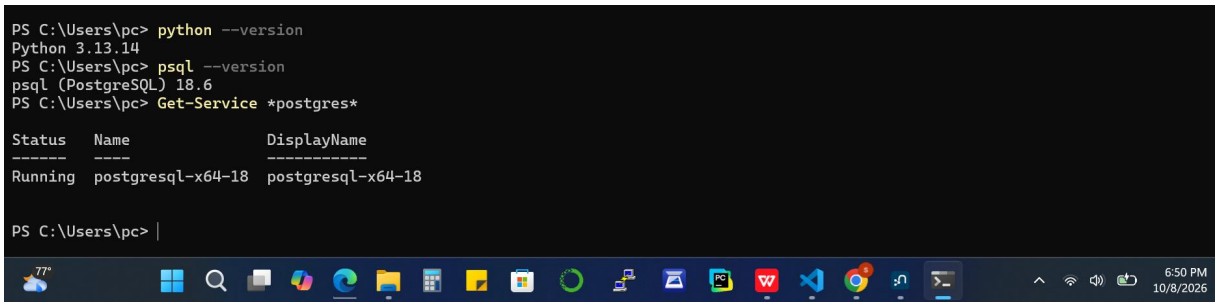


Figure 8. Python, psql and the running PostgreSQL service.

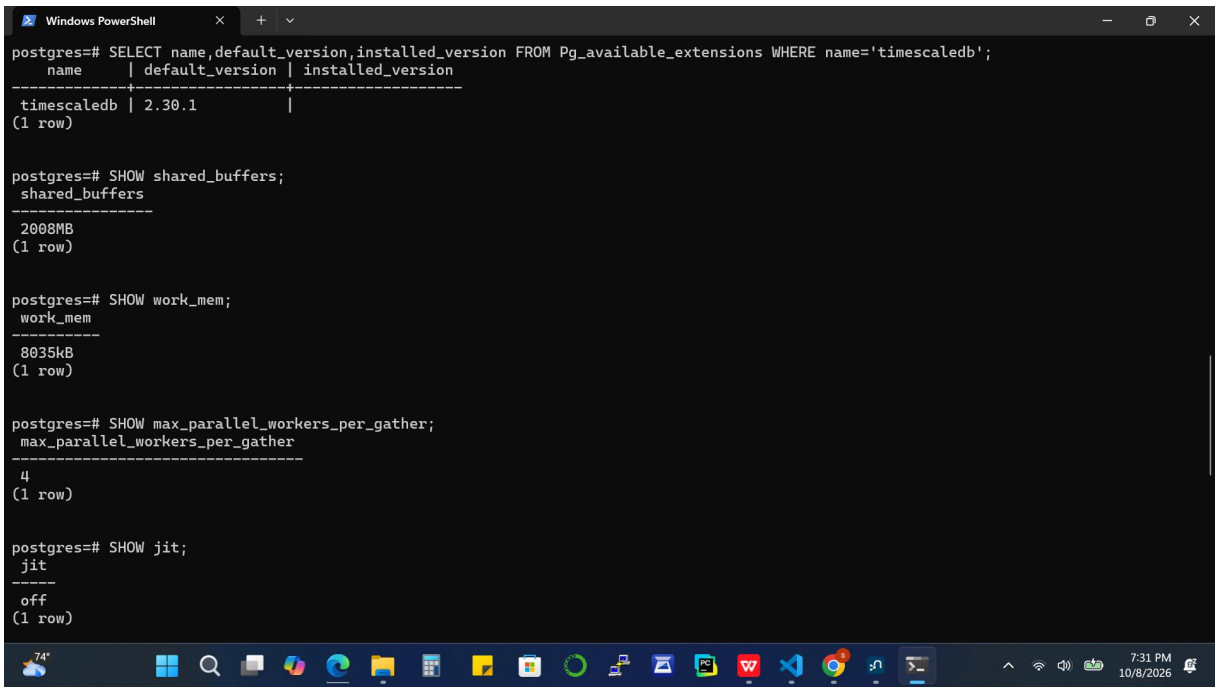


Figure 9. TimescaleDB version and the four PostgreSQL settings

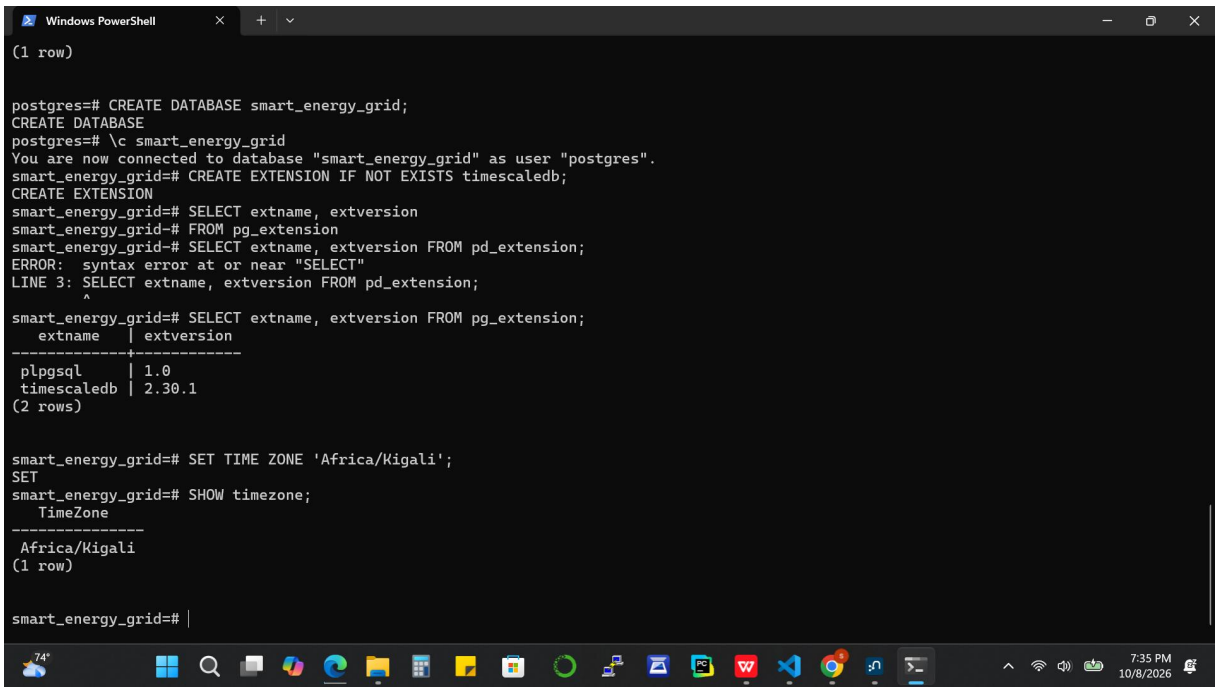


Figure 10. database created, timescaledb enabled (2.30.1) and session time zone Africa/Kigali.

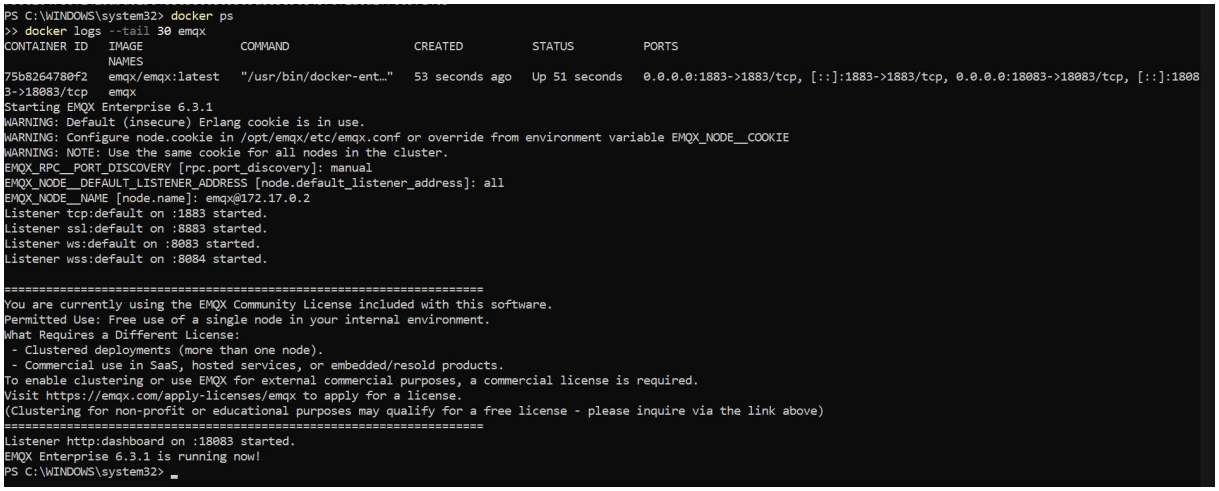


Figure 11. EMQX broker container running.

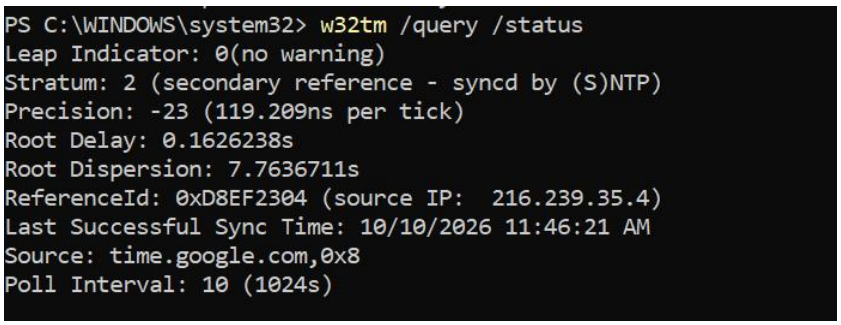


Figure 12. Clock synchronization on : `w32tm /query /status`, last successful sync 10/10/2026 11:46:21 AM

1.2 Create the database structures

sql/01_schema.sql creates energy_live (regular table), energy_readings (hypertable on event_time, one-day chunks) and meters. Both reading tables have the same eight columns and allow duplicates

(no unique constraint). load_meters.py imports smart_meter_simulator, calls meter_metadata() and inserts the 4,000 returned dictionaries.

Data dictionary

Table	Column	PostgreSQL type	Meaning	Unit
energy_live, energy_readings	meter_id	BIGINT	Meter identifier (1000000000 – 1000003999)	—
	event_time	TIMESTAMPTZ	Start of the 15-minute interval (hypertable time column)	CAT
	received_time	TIMESTAMPTZ	Simulated arrival time of the reading (not insertion time)	CAT
	power_kw	DOUBLE PRECISION	Average power over the interval	kW
	voltage_v	DOUBLE PRECISION	Supply voltage	V
	current_a	DOUBLE PRECISION	Current	A
	frequency_hz	DOUBLE PRECISION	Grid frequency	Hz
	energy_kwh	DOUBLE PRECISION	Energy used in the interval (≈ power_kw × 0.25 h)	kWh
meters	meter_id	BIGINT (primary key)	Meter identifier	—
	region	TEXT	Region 1 … Region 10 (400 meters each)	—
	customer_type	TEXT	residential, commercial or industrial	—
	base_power_kw	DOUBLE PRECISION	Typical base demand of the meter	kW
	power_factor	DOUBLE PRECISION	Power factor (0.95 for all meters)	none

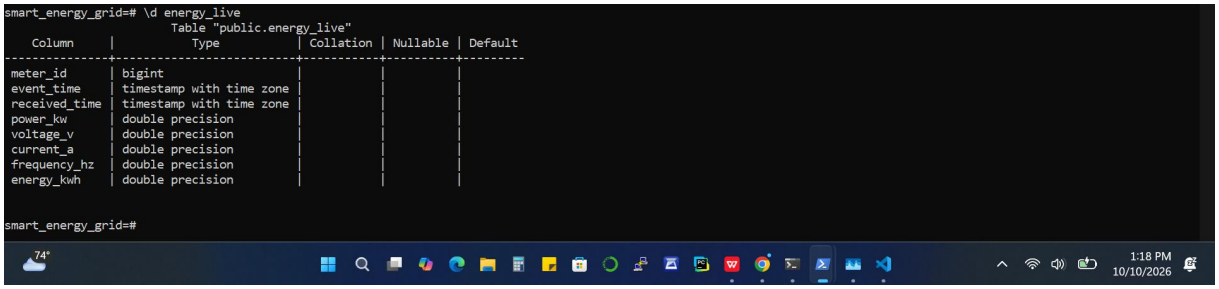


Figure 13. : `\d energy_live`.

```
smart_energy_grid=# \d energy_readings
                    Table "public.energy_readings"
    Column     |           Type           | Collation | Nullable | Default
---------------+--------------------------+-----------+----------+---------
 meter_id      | bigint                   |           |          |
 event_time    | timestamp with time zone |           | not null |
 received_time | timestamp with time zone |           |          |
 power_kw      | double precision         |           |          |
 voltage_v     | double precision         |           |          |
 current_a     | double precision         |           |          |
 frequency_hz  | double precision         |           |          |
 energy_kwh    | double precision         |           |          |
Indexes:
    "energy_readings_event_time_idx" btree (event_time DESC)


smart_energy_grid=#
```



Figure 14. : `\d energy_readings` with its event_time index.

```
smart_energy_grid=# \d meters
                      Table "public.meters"
    Column     |       Type       | Collation | Nullable | Default
---------------+------------------+-----------+----------+---------
 meter_id      | bigint           |           | not null |
 region        | text             |           |          |
 customer_type | text             |           |          |
 base_power_kw | double precision |           |          |
 power_factor  | double precision |           |          |
Indexes:
    "meters_pkey" PRIMARY KEY, btree (meter_id)


smart_energy_grid=#
```



Figure 15. : `\d meters`.

```
smart_energy_grid=# SELECT hypertable_name, column_name, column_type, time_interval
smart_energy_grid-# FROM timescaledb_information.dimensions
smart_energy_grid-# WHERE hypertable_name = 'energy_readings';
 hypertable_name | column_name |       column_type        | time_interval
-----------------+-------------+--------------------------+---------------
 energy_readings | event_time  | timestamp with time zone | 1 day
(1 row)


smart_energy_grid=#
```



Figure 16. : hypertable chunk interval = 1 day.

```
smart_energy_grid=# SELECT COUNT(*) AS total_meters FROM meters;
 total_meters
--------------
         4000
(1 row)


smart_energy_grid=# \
```


Figure 17. : `SELECT count(*) FROM meters` = 4000.

```
iot_assignment=# \d energy_live
                    Table "public.energy_live"
    Column     |           Type           | Collation | Nullable | Default
---------------+--------------------------+-----------+----------+---------
 meter_id      | bigint                   |           |          |
 event_time    | timestamp with time zone |           |          |
 received_time | timestamp with time zone |           |          |
 power_kw      | double precision         |           |          |
 voltage_v     | double precision         |           |          |
 current_a     | double precision         |           |          |
 frequency_hz  | double precision         |           |          |
 energy_kwh    | double precision         |           |          |

iot_assignment=# \d energy_readings
                   Table "public.energy_readings"
    Column     |           Type           | Collation | Nullable | Default
---------------+--------------------------+-----------+----------+---------
 meter_id      | bigint                   |           |          |
 event_time    | timestamp with time zone |           | not null |
 received_time | timestamp with time zone |           |          |
 power_kw      | double precision         |           |          |
 voltage_v     | double precision         |           |          |
 current_a     | double precision         |           |          |
 frequency_hz  | double precision         |           |          |
 energy_kwh    | double precision         |           |          |
Indexes:
    "energy_readings_event_time_idx" btree (event_time DESC)

iot_assignment=# \d meters
                     Table "public.meters"
    Column     |       Type       | Collation | Nullable | Default
---------------+------------------+-----------+----------+---------
 meter_id      | bigint           |           | not null |
 region        | text             |           |          |
 customer_type | text             |           |          |
 base_power_kw | double precision |           |          |
 power_factor  | double precision |           |          |
Indexes:
    "meters_pkey" PRIMARY KEY, btree (meter_id)

iot_assignment=#
```

Figure 18. Machine 1: `\d` of the three tables.

1.3 Capture and publish a pilot stream

pilot_publisher.py takes the first 2,000 dictionaries from iter_readings() and publishes each as JSON to energy/meters/{meter_id} with QoS 1 and retain = false. pilot_subscriber.py subscribes to energy/meters/# at QoS 1 (persistent session), parses the JSON and inserts the eight fields into energy_live; it is started first, on an empty table. run_pilot.py automates the whole run and prints the four counts.

Stage	Count	Target	Difference
Dictionaries taken from the generator	2,000	2,000	0
Messages acknowledged by the broker (QoS 1 PUBACK)	2,000	2,000	0
Messages received by the subscriber	2,000	2,000	0
Rows in energy_live after the final commit	2,000	2,000	0

No loss, redelivery, parse error or failed insert occurred: the subscriber summary reports received 2000, dup_flag 0, parse_errors 0, inserted 2000, insert_errors 0. Any deliberate generator duplicates inside the first 2,000 records are counted as separate messages and rows, as required.

```
PS C:\iot-project>    docker exec -it timescaledb psql -U postgres -d iot_assignment -c "TRUNCATE energy_live;" -c "SELECT count(*) FROM energy_live;"
TRUNCATE TABLE
 count
-------
     0
(1 row)
```

Figure 19. Machine 1: empty `energy_live` before the pilot

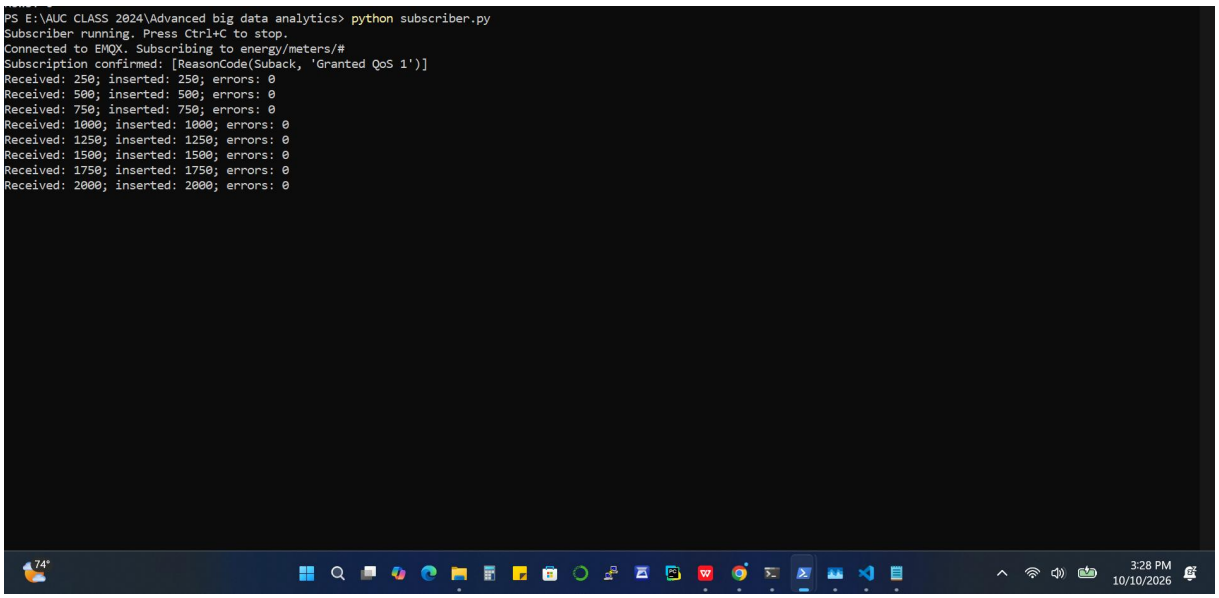


Figure 20. : subscriber started first, connected subscription QoS 1) and receiving: 2000 received, 2000 inserted, 0 errors.

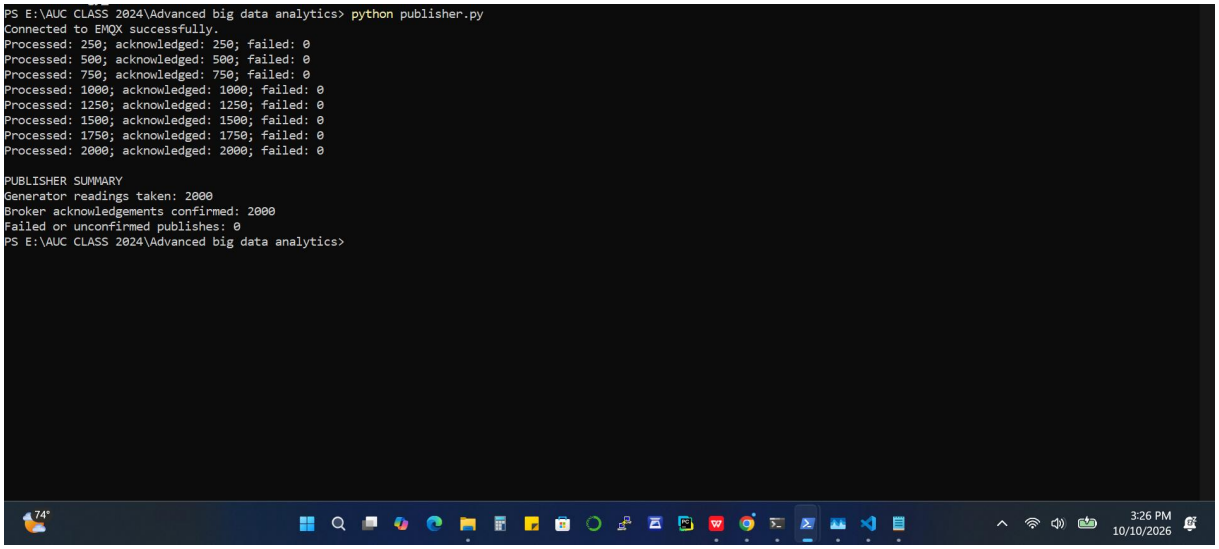


Figure 21. : publisher completion

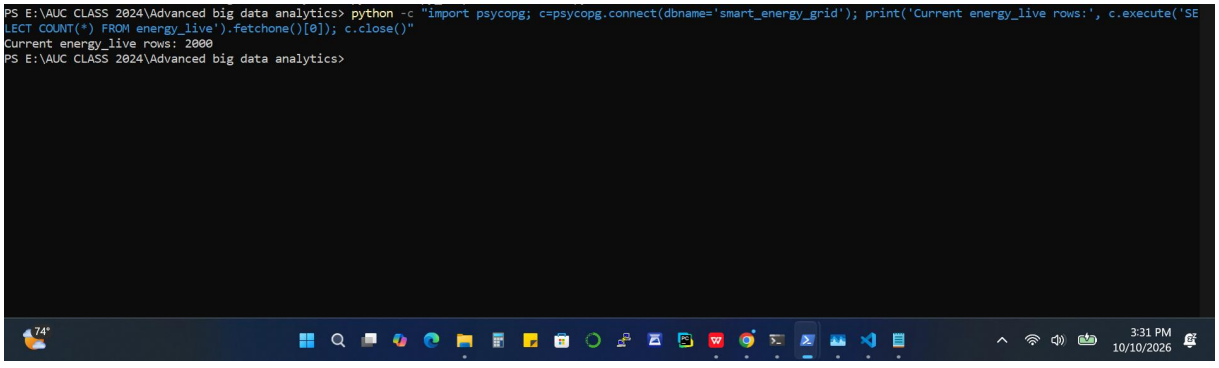


Figure 22. : stored count in energy_live = 2000.

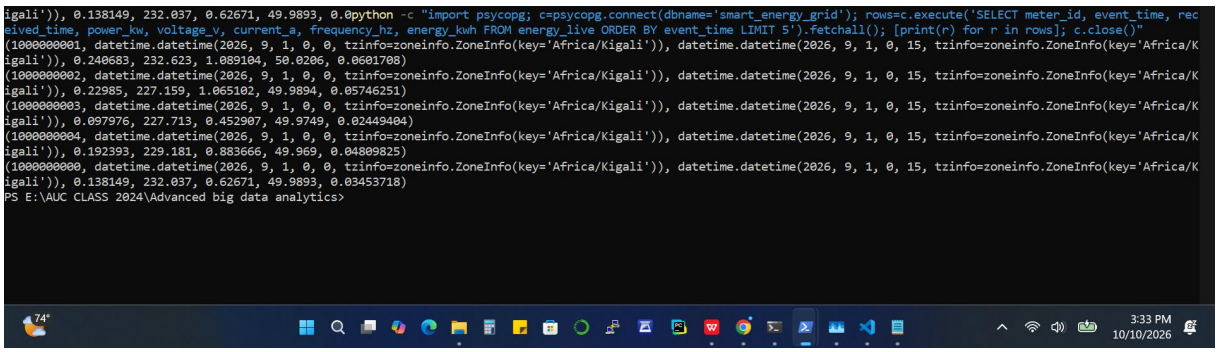


Figure 23. : sample rows from energy_live

2 Capture and prepare the historical dataset

2.1 Capture and load the complete stream

Route: CSV. capture_csv.py calls dataset_info(), starts a fresh iter_readings() iterator and streams every dictionary into readings.csv in batches while printing progress. load_csv.py bulk-loads the file with PostgreSQL COPY in 500,000-row batches inside one transaction into the empty energy_readings, duplicates included. The pilot rows stay in energy_live.

Check	Value
planned_readings (dataset_info)	10,752,000
expected emitted_readings	10,696,848
Dictionaries consumed by the generator	10,696,848
Data rows in readings.csv	10,696,848
Rows committed in energy_readings	10,696,848
Generation / capture time (CSV file)	196.3 s
Database load time (first COPY to final commit)	146.7 s
Total elapsed, generation start → final commit	343.0 s
Distinct meters / first / last event_time	4,000 / 2026-09-01 00:00:00+02 / 2026-09-28 23:45:00+02
Chunks	29 (one-day)
Size before the second index	table data 958 MB + indexes 85 MB = 1,042 MB (1,093,124,096 B)

The sizes above were taken right after the load (event_time index only). The final, comparable sizes with both indexes are given in 4.2 (1,357 MB). The intermediate CSV can be deleted after verification because capture_csv.py regenerates it identically.

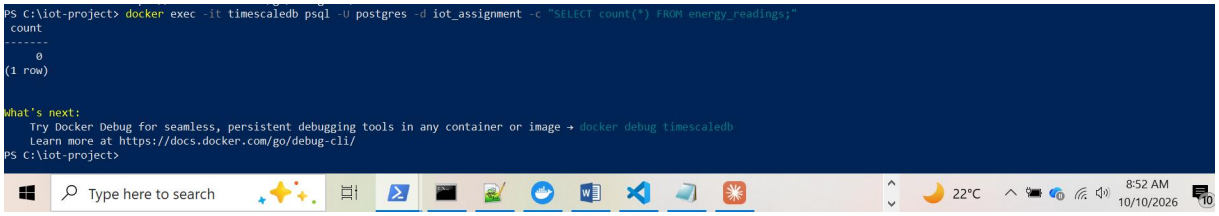


Figure 24. "energy_readings" is empty before the load

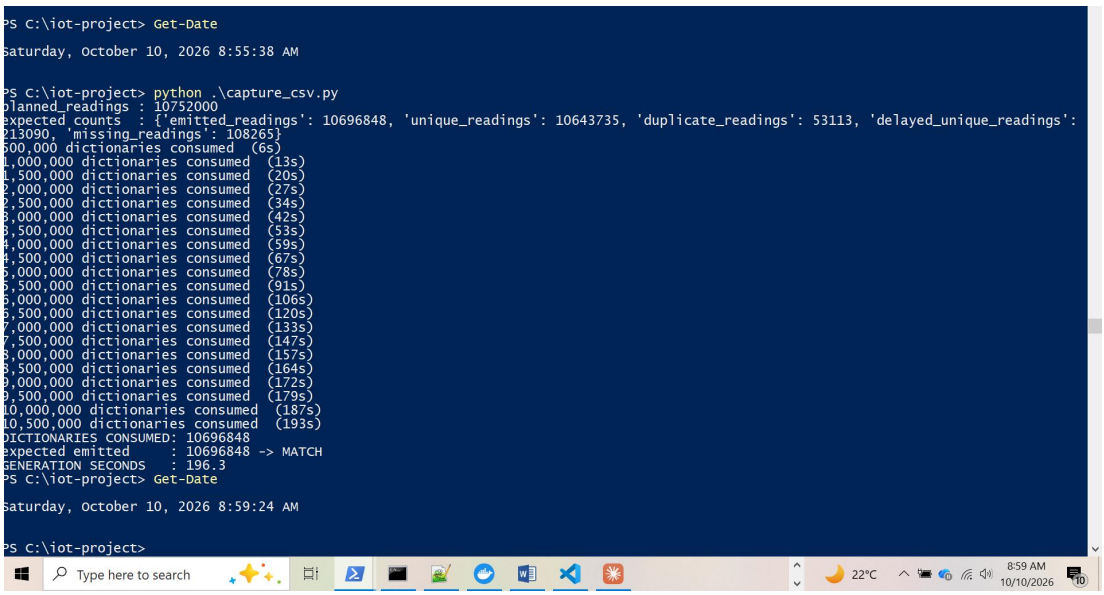


Figure 25. Capture/generation completed: progress and final dictionary count.



Figure 26. CSV data-row count.

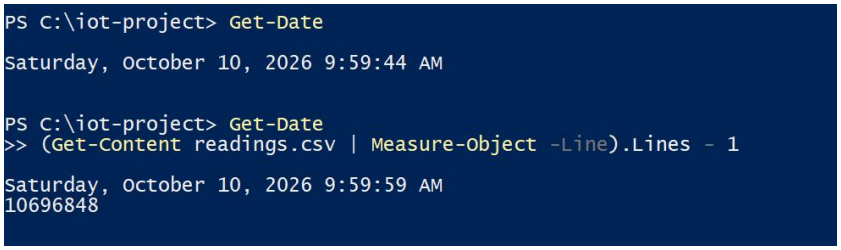


Figure 27. CSV data-row count.

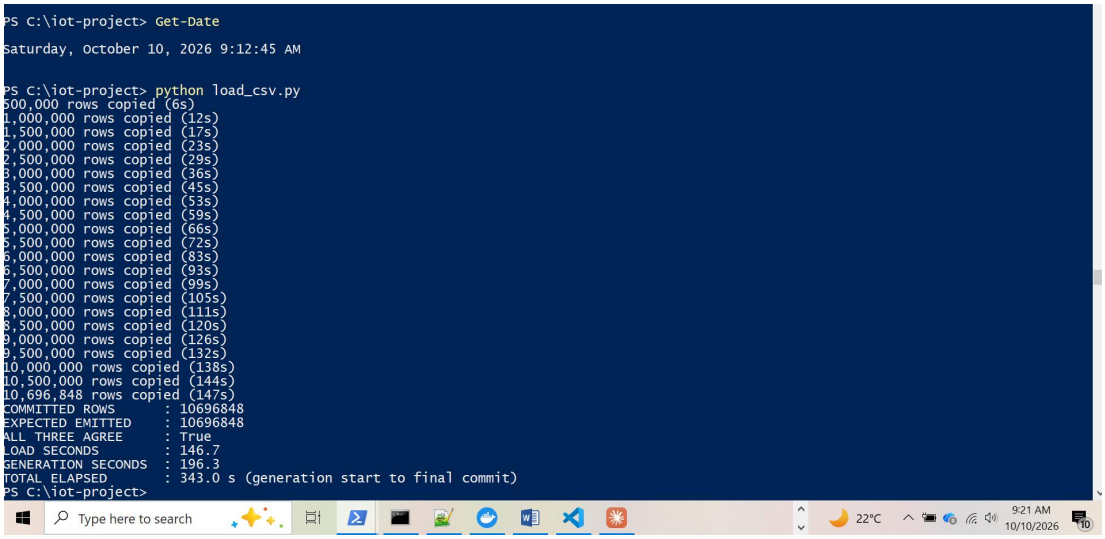


Figure 28. Bulk load completed: batches, final commit and load seconds.

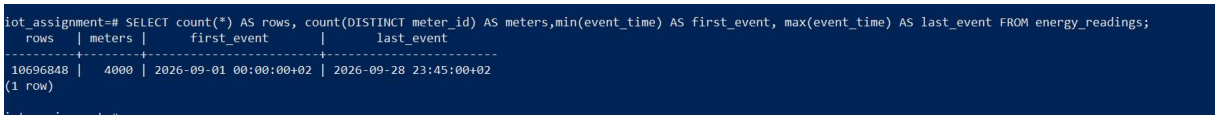


Figure 29. Database checks: rows, distinct meters, earliest and latest event_time.

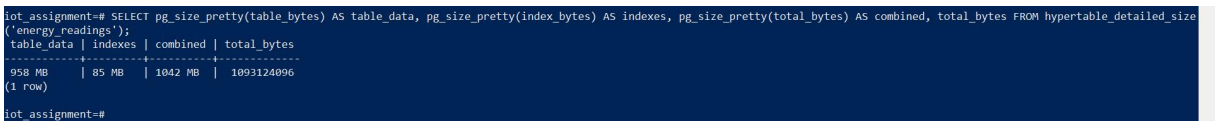


Figure 30. Table data, index and combined size of "energy_readings".

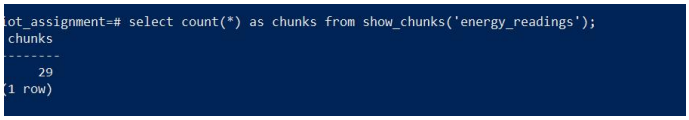


Figure 31. Chunk count of "energy_readings" = 29.

2.2 Investigate data quality

sql/04_quality.sql works on the uncleaned table. A unique reading is one (meter_id, event_time) pair; normal arrival is event_time + 15 minutes; delayed arrivals are 30–180 minutes after event_time.

Measure	Measured	Reference	Rate
Planned readings	10,752,000	10,752,000	—
Emitted rows	10,696,848	10,696,848	—
Unique readings (observed pairs)	10,643,735	10,643,735	completeness 98.9931 %
Extra copies (duplicates)	53,113	53,113	0.4990 % of unique pairs
Delayed unique readings	213,090	213,090	2.0020 % of unique pairs
Missing pairs	108,265	108,265	1.0069 % of planned
Unique + extra copies = emitted	10,643,735 + 53,113 = 10,696,848	OK	
Planned − unique = missing	10,752,000 − 10,643,735 = 108,265	OK	

Formulas:

- missing % = 100 × 108,265 / 10,752,000;
- duplicate % = 100 × 53,113 / 10,643,735;
- delayed % = 100 × 213,090 / 10,643,735;
- completeness = 100 × 10,643,735 / 10,752,000.

The five meters with the most missing readings are shown in the last screenshot.

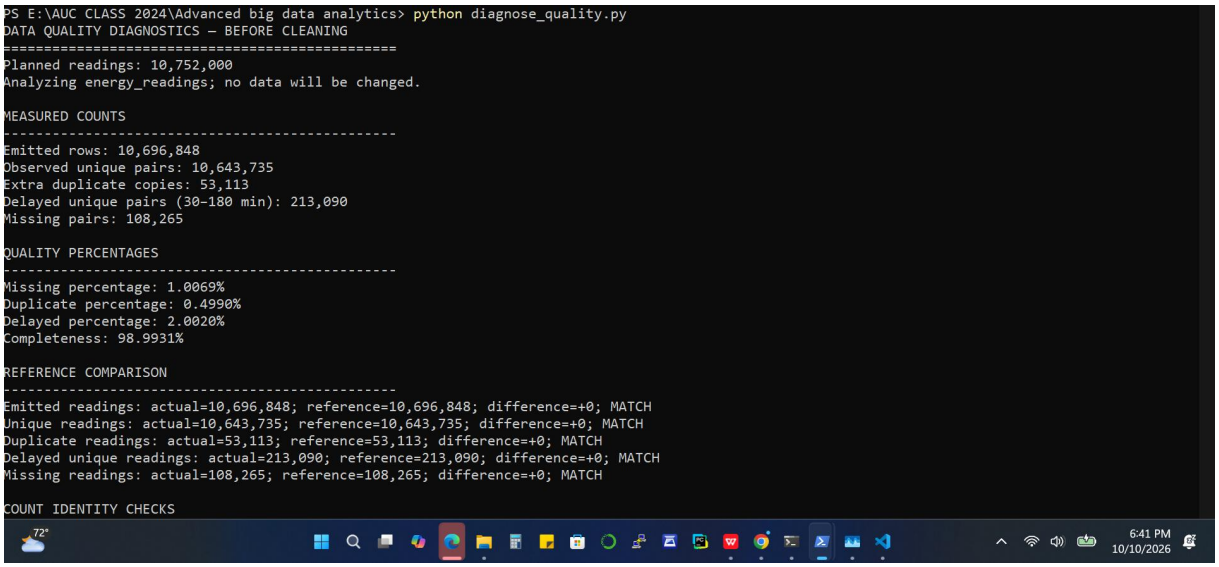


Figure 32. : data-quality diagnostics before cleaning: measured counts, percentages, comparison with the reference table

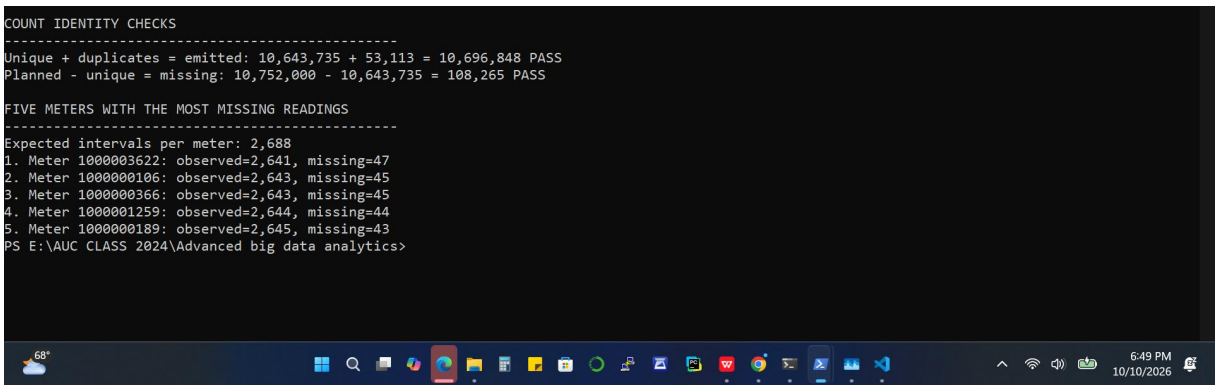


Figure 33. : count identity checks

2.3 Clean and preserve a consistent benchmark

clean.py groups rows sharing (meter_id, event_time) and compares received_time, power_kw, voltage_v, current_a, frequency_hz and energy_kwh. It first counts conflicts (groups whose six values differ): **0 conflicts**, so every extra row is an exact copy. It keeps one row per pair (the physical row with the lowest ctid) and deletes the others; missing intervals stay as gaps. The script is repeatable and verifies the result.

The final count equals expected_counts.unique_readings (10,643,735) and no duplicate pair remains (0 groups with more than one row).

	Rows	Sum energy_kwh
Before cleaning	10,696,848	3,872,863.893179
After cleaning	10,643,735	3,853,704.346816
Removed	53,113	19,159.546363

Why missing readings must not become zero. A missing reading means no measurement arrived (communication fault, meter outage, or a reading that never reached the broker), not that the customer used no electricity. Filling zeros would understate consumption and demand, distort

averages and completeness, teach a forecast model false zero-demand hours and hide faults that operations must repair. Gaps stay visible and are reported as completeness.

```
Saturday, October 10, 2026 10:23:48 AM
BEFORE: rows=10,696,848  sum_energy_kwh=3872863.893179
CONFLICTING DUPLICATE PAIRS: 0
day window 2026-08-31 22:00Z removed 1981
day window 2026-09-01 22:00Z removed 1900
day window 2026-09-02 22:00Z removed 1921
day window 2026-09-03 22:00Z removed 1921
day window 2026-09-04 22:00Z removed 1933
day window 2026-09-05 22:00Z removed 1948
day window 2026-09-06 22:00Z removed 1915
day window 2026-09-07 22:00Z removed 1821
day window 2026-09-08 22:00Z removed 1798
day window 2026-09-09 22:00Z removed 1885
day window 2026-09-10 22:00Z removed 1949
day window 2026-09-11 22:00Z removed 1927
day window 2026-09-12 22:00Z removed 1962
day window 2026-09-13 22:00Z removed 1835
day window 2026-09-14 22:00Z removed 1961
day window 2026-09-15 22:00Z removed 1911
day window 2026-09-16 22:00Z removed 1909
day window 2026-09-17 22:00Z removed 1910
day window 2026-09-18 22:00Z removed 1922
day window 2026-09-19 22:00Z removed 1914
day window 2026-09-20 22:00Z removed 1896
day window 2026-09-21 22:00Z removed 1835
day window 2026-09-22 22:00Z removed 1814
day window 2026-09-23 22:00Z removed 1873
day window 2026-09-24 22:00Z removed 1864
day window 2026-09-25 22:00Z removed 1881
day window 2026-09-26 22:00Z removed 1840
day window 2026-09-27 22:00Z removed 1887
day window 2026-09-28 22:00Z removed 0
day window 2026-09-29 22:00Z removed 0
TOTAL REMOVED: 53113
AFTER : rows=10,643,735  sum_energy_kwh=3853704.346816
REMAINING DUPLICATE PAIRS: 0
MATCHES expected unique_readings (10,643,735): True
Rows removed: 53113  Energy removed (kWh): 19159.546363248955
Saturday, October 10, 2026 10:26:22 AM
```

Figure 34. Machine 1: cleaning completion with before/after row counts, summed energy and duplicate check.

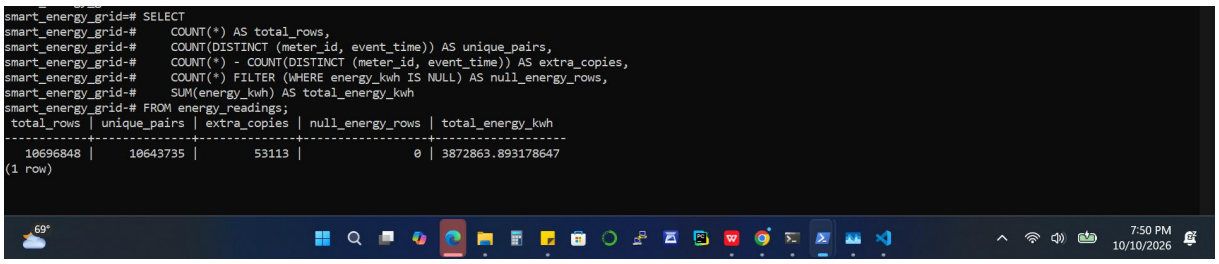


Figure 35. : counts and summed energy before cleaning (10,696,848 rows, 53,113 extra copies, 3,872,863.89 kWh).

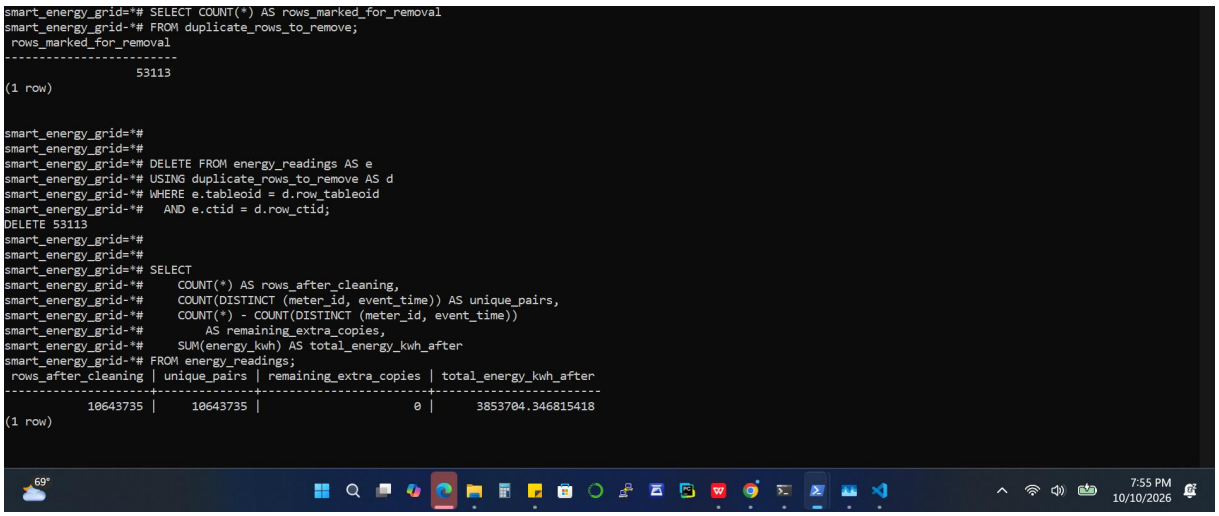


Figure 36. : deletion of the 53,113 identical copies.

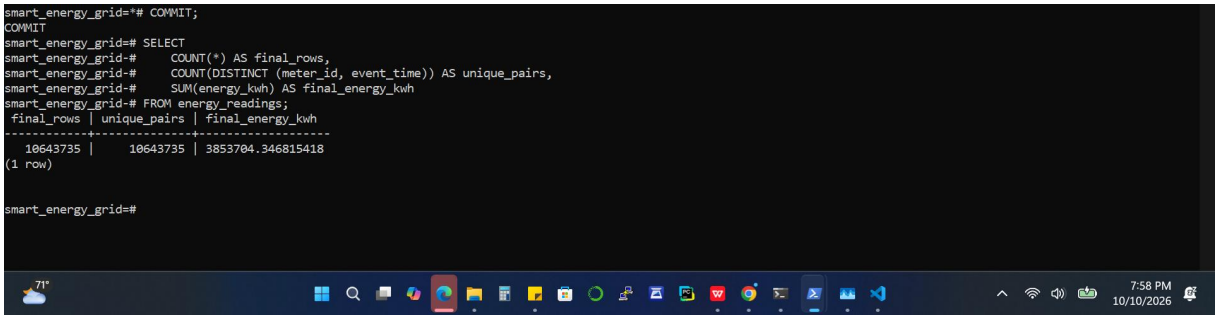


Figure 37. : after cleaning: 10,643,735 rows, 0 extra copies, 3,853,704.35 kWh.

3 Compare chunk intervals

3.1 Create and verify three configurations

sql/02_chunk_tables.sql creates energy_readings_3h (3-hour chunks) and energy_readings_week (7-day chunks) with the same eight columns. sql/02b_copy_chunk_tables.sql copies the cleaned rows into both, and sql/02d_indexes.sql creates the same two non-unique B-tree indexes on all three tables: (event_time) and (meter_id, event_time), both ascending. ANALYZE is run after loading.

Table	Chunk interval	Chunks	Rows	Meters	First / last event_time
energy_readings	1 day	29	10,643,735	4,000	2026-09-01 00:00 / 2026-09-28 23:45
energy_readings_3h	3 hours	225	10,643,735	4,000	same
energy_readings_week	7 days	5	10,643,735	4,000	same

Chunk counts are fixed by TimescaleDB's UTC-aligned chunk boundaries (28 days of data span 29 one-day chunks, 225 three-hour chunks and 5 seven-day chunks). They were confirmed on both machines (see the chunk-count screenshot). Numeric sums agree within the 0.001 tolerance (the tolerance screenshot shows within_tolerance = t for all three tables; the largest difference was about 1.5e-7).

```
PS C:\iot-project> docker exec -it timescaledb psql -U postgres -d iot_assignment -c "\d energy_readings"
                    Table "public.energy_readings"
    Column     |           Type           | Collation | Nullable | Default
---------------+--------------------------+-----------+----------+---------
 meter_id      | bigint                   |           |          |
 event_time    | timestamp with time zone |           | not null |
 received_time | timestamp with time zone |           |          |
 power_kw      | double precision         |           |          |
 voltage_v     | double precision         |           |          |
 current_a     | double precision         |           |          |
 frequency_hz  | double precision         |           |          |
 energy_kwh    | double precision         |           |          |
Indexes:
    "energy_readings_event_time_idx" btree (event_time DESC)
Number of child tables: 29 (Use \d+ to list them.)


What's next:
    Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug timescaledb
    Learn more at https://docs.docker.com/go/debug-cli/
PS C:\iot-project>
```



Figure 38. Structure of the baseline hypertable "energy_readings"including indexes.

```
iot_assignment=# WITH s AS (
  SELECT 'energy_readings' AS tbl, sum(power_kw) p, sum(energy_kwh) e FROM energy_readings
  UNION ALL SELECT 'energy_readings_3h', sum(power_kw), sum(energy_kwh) FROM energy_readings_3h
  UNION ALL SELECT 'energy_readings_week', sum(power_kw), sum(energy_kwh) FROM energy_readings_week)
SELECT tbl,
       abs(p - first_value(p) OVER ()) AS power_diff,
       abs(e - first_value(e) OVER ()) AS energy_diff,
       (abs(p - first_value(p) OVER ()) <= 0.001
        AND abs(e - first_value(e) OVER ()) <= 0.001) AS within_tolerance
FROM s;
        tbl          |       power_diff       |      energy_diff       | within_tolerance
---------------------+------------------------+------------------------+------------------
 energy_readings_week |                      0 |                      0 | t
 energy_readings      |  1.210719347000122e-07 |   1.51805579662323e-07 | t
 energy_readings_3h   | 1.1175870895385742e-07 | 1.4388933777809143e-07 | t
(3 rows)
```

Figure 39. Machine 1: all three tables within the 0.001 tolerance (within_tolerance = t).

```
smart_energy_grid=#
smart_energy_grid=# SELECT
smart_energy_grid-#     hypertable_name,
smart_energy_grid-#     column_name,
smart_energy_grid-#     time_interval
smart_energy_grid-# FROM timescaledb_information.dimensions
smart_energy_grid-# WHERE hypertable_name IN (
smart_energy_grid(#     'energy_readings',
smart_energy_grid(#     'energy_readings_3h',
smart_energy_grid(#     'energy_readings_week'
smart_energy_grid(# )
smart_energy_grid-# ORDER BY hypertable_name;
   hypertable_name    | column_name | time_interval
----------------------+-------------+---------------
 energy_readings      | event_time  | 1 day
 energy_readings_3h   | event_time  | 03:00:00
 energy_readings_week | event_time  | 7 days
(3 rows)


smart_energy_grid=#
```



Figure 40. : chunk interval of each hypertable (1 day, 03:00:00, 7 days).

```
smart_energy_grid=# SELECT
smart_energy_grid-#     hypertable_name,
smart_energy_grid-#     COUNT(*) AS chunk_count
smart_energy_grid-# FROM timescaledb_information.chunks
smart_energy_grid-# WHERE hypertable_name IN (
smart_energy_grid(#     'energy_readings',
smart_energy_grid(#     'energy_readings_3h',
smart_energy_grid(#     'energy_readings_week'
smart_energy_grid(# )
smart_energy_grid-# GROUP BY hypertable_name
smart_energy_grid-# ORDER BY hypertable_name;
   hypertable_name    | chunk_count
----------------------+-------------
 energy_readings      |          29
 energy_readings_3h   |         225
 energy_readings_week |           5
(3 rows)


smart_energy_grid=#
```



Figure 41. : chunk counts: 29, 225 and 5.

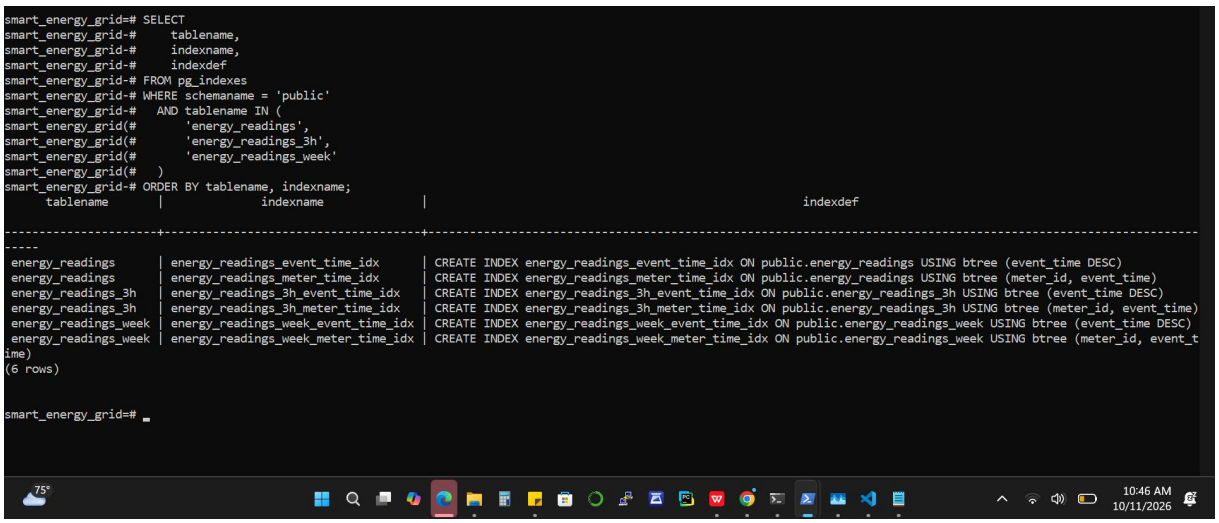


Figure 42. : matching indexes on the three tables "event_time DESC and (meter_id, event_time)".

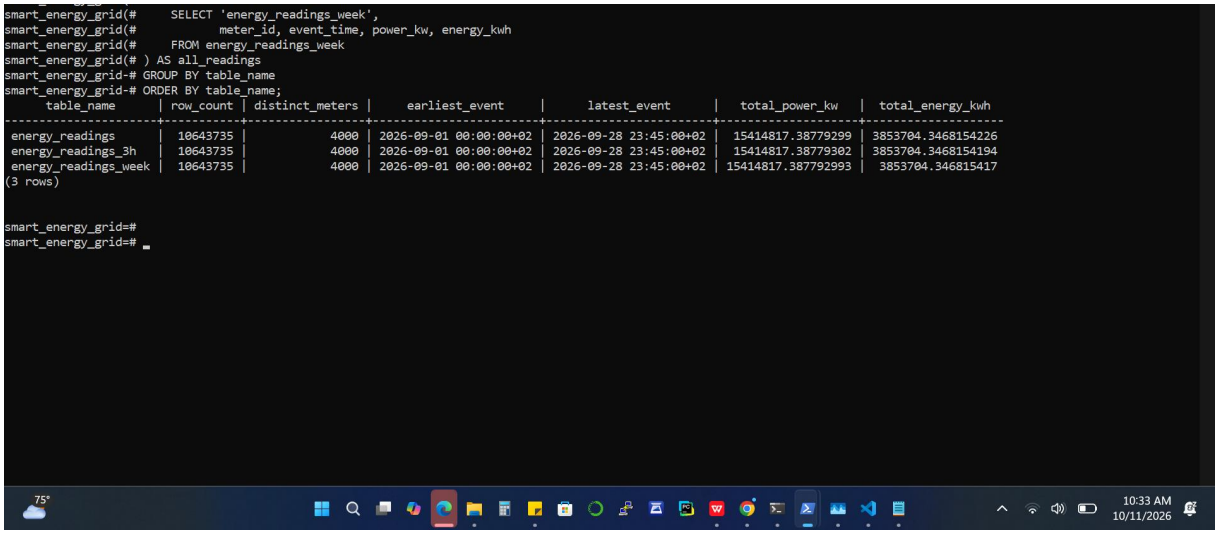


Figure 43. : matching data checks (rows 10,643,735, 4,000 meters, dates, sums) on all three tables.

3.2 Implement and time the five workloads

benchmark.py runs Q1–Q5 on the three tables from the same Python client (psycopg2), one table after another, with the same session settings. For every query and table: 1 warm-up and 5 measured executions; the timer (time.perf_counter()) starts at statement submission and stops after the last result row is received. Results are compared across tables (tolerance 0.001). PostgreSQL is not restarted and caches are not cleared, so **all numbers are warm-cache measurements**. Policies and loaders were paused. Machine: DESKTOP-N3TT2CL; client: Python 3.12.10 + psycopg2; timing tool: time.perf_counter. Session start (CAT) 2026-10-11 11:43:21.

Query	1 day (29 chunks)	3 hours (225 chunks)	1 week (5 chunks)	Fastest
Q1	244.22 (233.57–305.93)	258.04 (198.37–319.18)	291.21 (239.35–314.41)	1 day
Q2	313.16 (260.71–330.16)	301.58 (271.94–313.76)	430.32 (375.59–483.44)	3 hours
Q3	1,818.38 (1,421.81–2,602.39)	3,372.46 (2,753.08–4,399.14)	1,371.34 (1,201.13–1,613.08)	1 week
Q4	4.23 (3.93–5.11)	8.03 (7.38–8.57)	3.02 (2.87–4.53)	1 week
Q5	1,068.95 (1,033.32–	1,089.72 (968.97–	1,005.32 (825.86–	1 week

	1,122.01)	1,271.38)	1,087.84)	

Cells: median (minimum–maximum) in milliseconds over 5 measured runs. Session seconds from first warm-up to last measured result per query/table are in results/group_machine_benchmark_3_2.csv (e.g. Q3: 11.6 s, 24.5 s, 9.0 s).

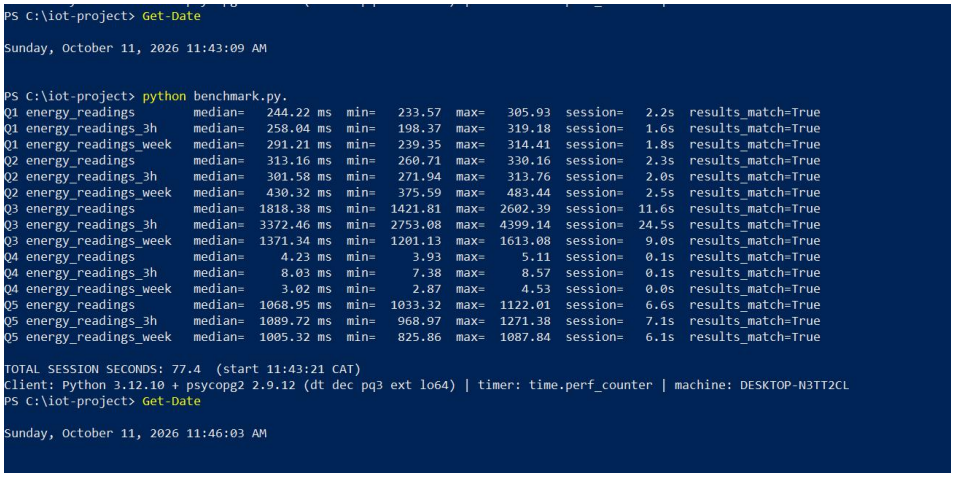


Figure 44. Completed timing comparison (median, min, max per query and table).

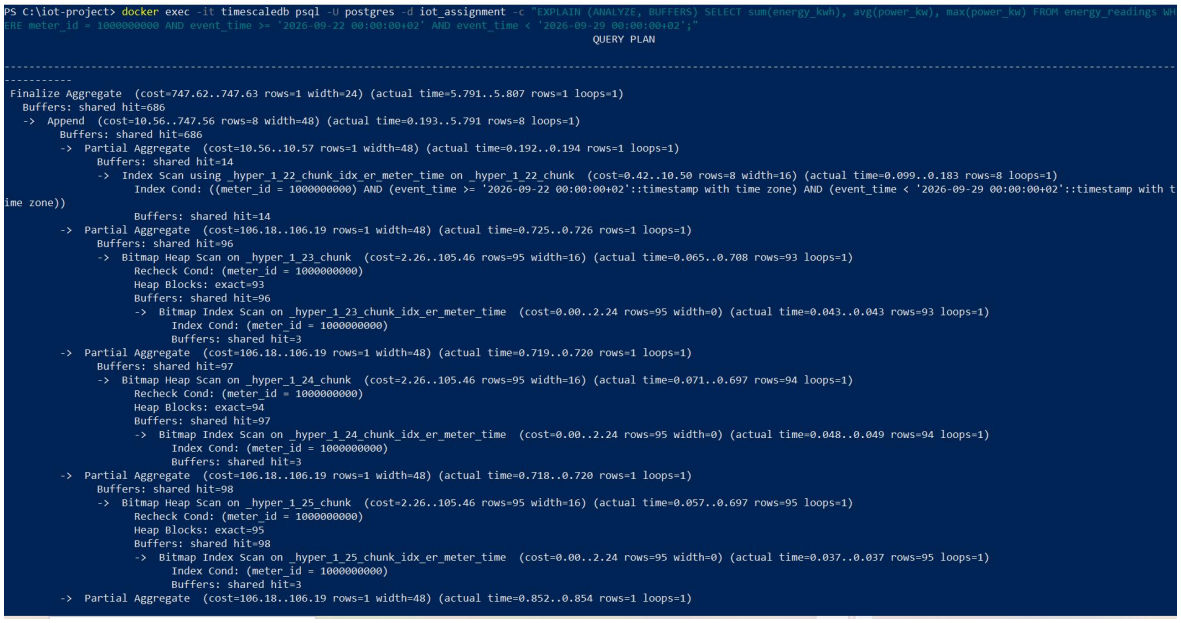


Figure 45. Execution plan with actual timing and buffers (Q4 on the one-day table).

3.3 Explain the chunk results

Observation = what was measured; explanation = the likely cause. Percentages compare each median with the fastest interval.

Workload	Fastest	Median comparison (ms)	Observation → explanation
Q1 hourly average, 21 Sep	1 day	244.2 vs 258.0 (3 h, +5.7 %) vs 291.2 (week, +19.2 %)	One CAT day touches 2 one-day chunks, about 9 three-hour chunks and 1 week-chunk. Day chunks match the filter best; the week chunk must read and discard other days of its range, the 3-hour set adds per-chunk overhead.
Q2 top ten 15-minute totals, 22–28 Sep	3 hours	301.6 vs 313.2 (1 day, +3.8 %) vs 430.3 (week, +42.7 %)	Observed difference between 3 h and 1 day is small (their min–max ranges overlap), so no firm winner. The week table is clearly slower: the 7-day filter touches 2 big chunks that also hold rows outside the range, so more rows are read and filtered before the sort.
Q3 total energy per meter, full period	1 week	1,371.3 vs 1,818.4 (1 day, +32.6 %) vs 3,372.5 (3 h, +145.9 %, 2.5×)	A full scan with no time filter reads every row in every case; the cost that differs is per-chunk work: 225 chunks produce 225 partial aggregates over 4,000 meter groups to merge, 5 chunks only 5.
Q4 one meter, 22–28 Sep	1 week	3.02 vs 4.23 (1 day, +40 %) vs 8.03 (3 h, +166 %)	Index-lookup query, a few milliseconds. The validation-run plans show 8 chunks scanned (1 day), 57 (3 h) and 2 (week) with planning times of 2.3, 19.1 and 2.2 ms: time is dominated by planning and one index descent per chunk, so fewer chunks win.
Q5 full scan aggregate	1 week	1,005.3 vs 1,069.0 (1 day, +6.3 %) vs 1,089.7 (3 h, +8.4 %)	All rows must be read in every layout, so differences are within about 8 % and overlap in min–max; chunking has little effect.

Replication on (native PostgreSQL 18.6, TimescaleDB 2.30.1, psycopg 3.3.6, same procedure: 1 warm-up + 5 measured runs, warm cache). Medians in ms:

Query	1 day	3 hours	1 week	Fastest on	Fastest on machine 1
Q1	451.3	407.0	474.3	3 hours	1 day
Q2	766.1	957.6	967.7	1 day	3 hours
Q3	3,025.8	13,405.2	4,637.1	1 day	1 week
Q4	15.66	47.69	5.49	1 week	1 week
Q5	2,829.4	2,908.5	2,393.4	1 week	1 week

Summary. Short time-filtered queries (Q1, Q4) favour chunks that fit the filter or have few chunks to plan; whole-table queries (Q3, Q5) favour fewer, larger chunks; the three-hour layout is never clearly best and is the worst for Q3 and Q4 because of 225 chunks. These effects come from the number of chunks planned and scanned, supported by the chunk counts in 3.1 and the plans; the causes themselves are our interpretation of these measurements, not separately measured.

4 Compare compression strategies

4.1 Create the two compressed configurations

Chosen chunk interval: 1 day (in 3.2 it was fastest for Q1 and best or second-best on every workload, never the slowest; it matches the daily reporting window; 29 chunks stay manageable). sql/03_compressed_tables.sql creates energy_time_compressed and energy_meter_compressed as one-day hypertables with the same eight columns and the same two indexes. Column store settings: both use order by = 'event_time DESC'; energy_time_compressed has no segment by, energy_meter_compressed has segment by = 'meter_id'. Each table has a compression policy with compress_after => INTERVAL '48 hours'; both policies (jobs 1000 and 1001) were paused with alter_job(..., scheduled => false) until the end of 4.3.

load_compressed.py records one cutoff before loading = experiment start − 48 h: **2026-10-10 21:30:05+02** (experiment start 2026-10-12 21:30:05 CAT), the same for both tables. It copies the cleaned data one chunk at a time, manually compresses a chunk after it is complete if its end is before the cutoff, and times each compression separately from loading. All 29 chunks end before the cutoff, so all 29 were compressed in both tables. Load time: 205.1 s (time-ordered table) and 139.1 s (meter-segmented table), excluding compression.

```
          create_hypertable
-------------------------------------
 (4,public,energy_time_compressed,t)
(1 row)

CREATE INDEX
CREATE TABLE
          create_hypertable
---------------------------------------
 (5,public,energy_meter_compressed,t)
(1 row)

CREATE INDEX
ALTER TABLE
ALTER TABLE
 job_time
----------
     1000
(1 row)

 job_meter
-----------
      1001
(1 row)


What's next:
```

Figure 46. Creation of the two compressed hypertables.

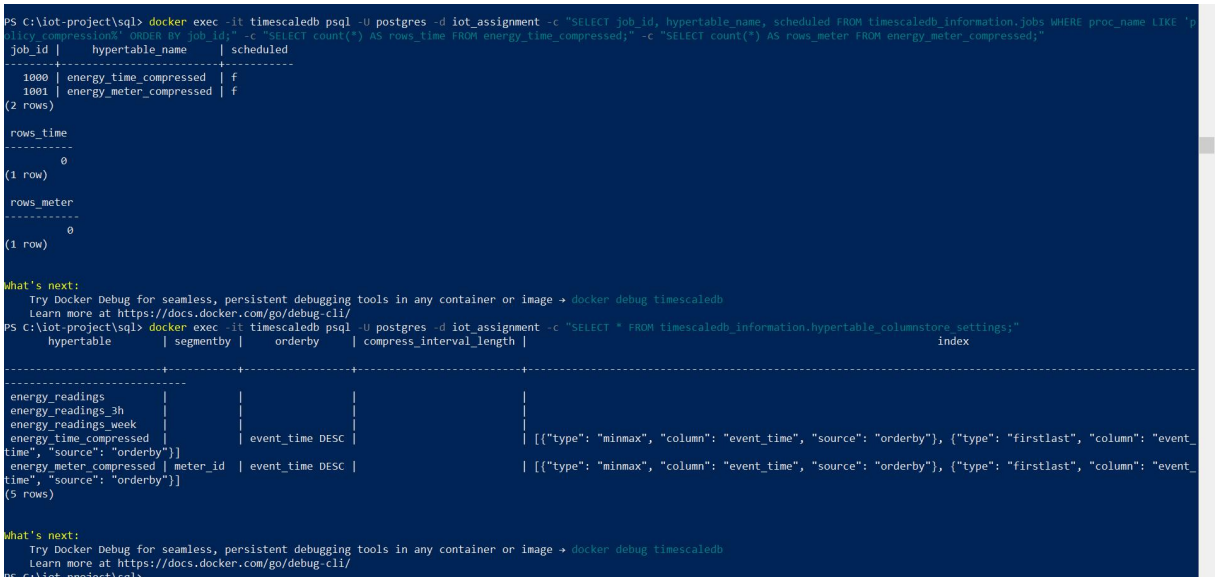


Figure 47. Policies (jobs 1000 and 1001, scheduled = f), column store settings (order by event_time DESC; segment by meter_id) and 0 rows before loading.

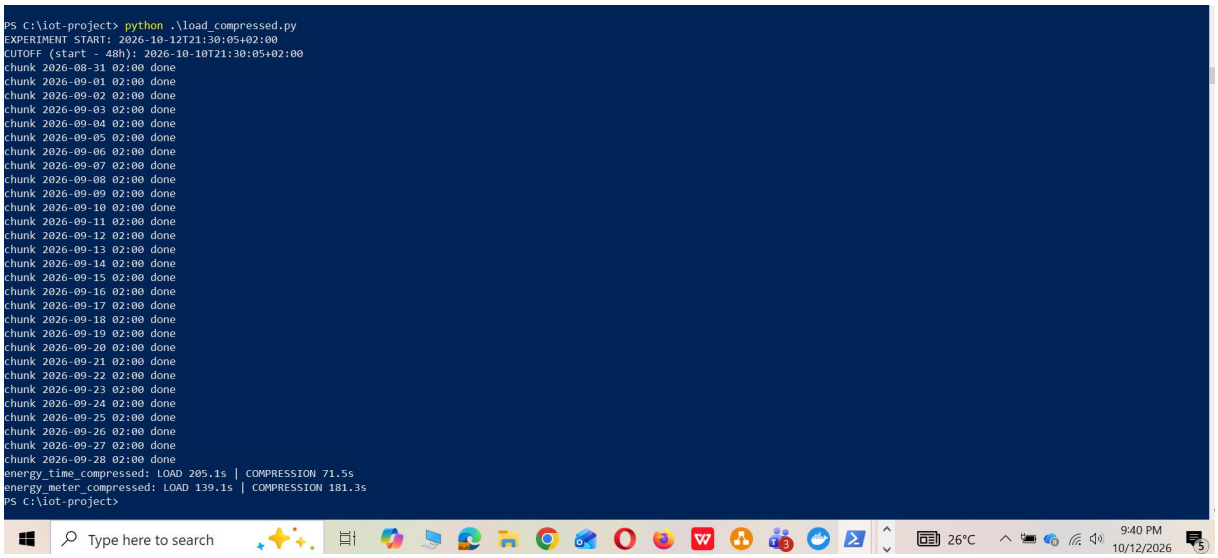


Figure 48. Chunk-by-chunk load and compression output with times.

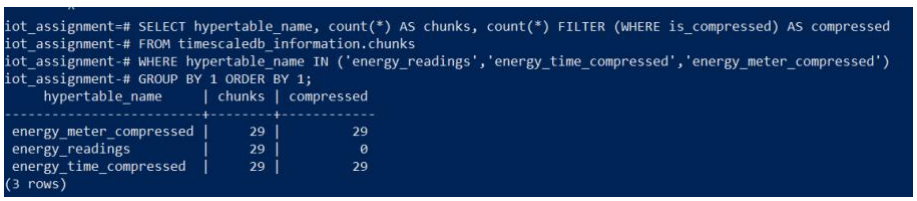


Figure 49. Chunk counts after loading: both compressed tables 29 of 29 chunks compressed; baseline 0.

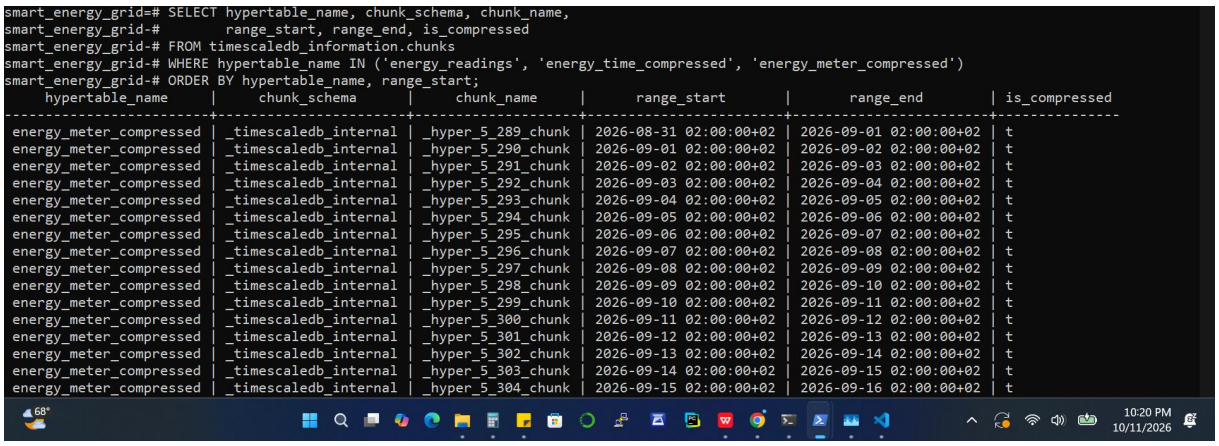


Figure 50. : compressed-chunk list: the same ranges are compressed in both tables.

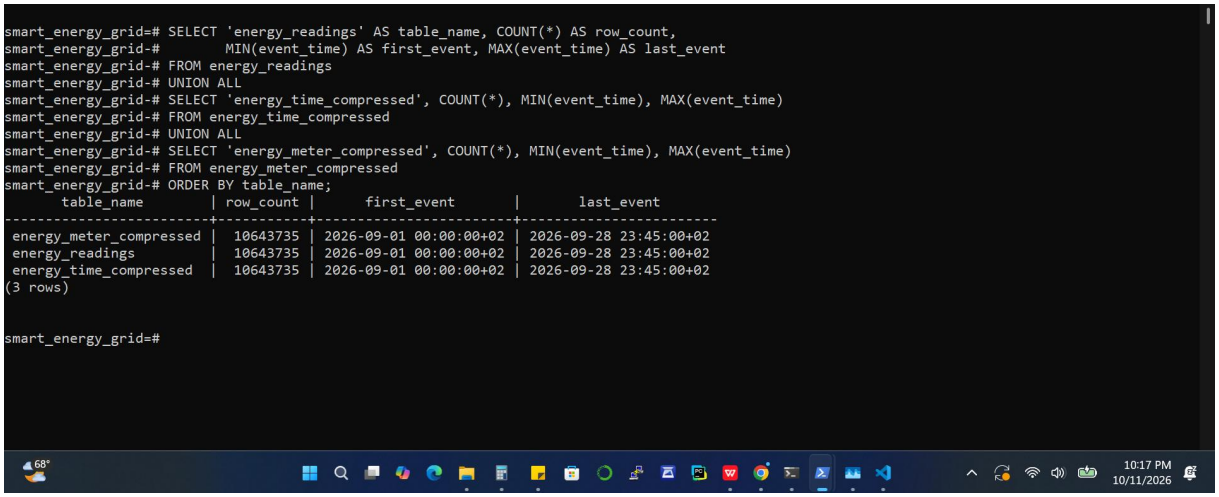


Figure 51. : data checks repeated on the compressed tables and the baseline (rows, meters, dates).

4.2 Measure storage and compression cost

Baseline = energy_readings (uncompressed, 1-day chunks). Sizes include all chunks and indexes (hypertable_detailed_size).

	Table data	Index	TOAST	Total	Ratio	Saved	Compression time	Compressed chunks	Rows in compressed chunks
Baseline (uncompressed)	951 MB	406 MB	—	1,357 MB (1,422,598,144 B)	1.00	0 %	—	0 / 29	0 %
Time-only (no segmentby)	15 MB	944 kB	423 MB	439 MB (459,980,800 B)	3.09	67.7 %	71.5 s	29 / 29	100 %
Segment by meter_id	27 MB	5,584 kB	469 MB	502 MB (525,918,208 B)	2.70	63.0 %	181.3 s	29 / 29	100 %

Ratio = baseline total / compressed total (1,422,598,144 / 459,980,800 = 3.09; / 525,918,208 = 2.70). Saved % = 100 × (baseline − compressed) / baseline. Compression cost = sum of the 29 measured per-chunk compression times, loading excluded.

Replication on (same SQL, 1-day chunks, both tables 29 of 29 chunks and 100 % of rows compressed):

	Total size	**Ratio**	**Saved**	**Compression time**
Baseline (uncompressed)	1,426,841,600 B (1,361 MiB)	1.00	0 %	—
Time-only	460,038,144 B (439 MiB)	3.10	67.76 %	35.34 s (1.22 s per chunk)
Segment by meter_id	526,016,512 B (502 MiB)	2.71	63.13 %	39.82 s (1.37 s per chunk)

The ratios and percentages agree with machine 1 within 0.01 (ratio) and 0.15 percentage points (saved) because compression ratios depend on the data, not on the machine; compression time is machine-dependent (machine 1 needed 71.5 s and 181.3 s), so absolute seconds must not be compared across machines.

```
iot_assignment=# SELECT 'energy_readings' AS tbl, pg_size_pretty(table_bytes) AS data, pg_size_pretty(index_bytes) AS idx,
iot_assignment-#        pg_size_pretty(toast_bytes) AS toast, pg_size_pretty(total_bytes) AS total, total_bytes
iot_assignment-# FROM hypertable_detailed_size('energy_readings')
iot_assignment-# UNION ALL SELECT 'energy_time_compressed', pg_size_pretty(table_bytes), pg_size_pretty(index_bytes),
iot_assignment-#        pg_size_pretty(toast_bytes), pg_size_pretty(total_bytes), total_bytes FROM hypertable_detailed_size('energy_time_compressed')
iot_assignment-# UNION ALL SELECT 'energy_meter_compressed', pg_size_pretty(table_bytes), pg_size_pretty(index_bytes),
iot_assignment-#        pg_size_pretty(toast_bytes), pg_size_pretty(total_bytes), total_bytes FROM hypertable_detailed_size('energy_meter_compressed');
          tbl           |  data  |   idx   |  toast  |  total  | total_bytes
------------------------+--------+---------+---------+---------+-------------
 energy_readings        | 951 MB | 406 MB  | 0 bytes | 1357 MB |  1422598144
 energy_time_compressed | 15 MB  | 944 kB  | 423 MB  | 439 MB  |   459980800
 energy_meter_compressed | 27 MB  | 5584 kB | 469 MB  | 502 MB  |   525918208
(3 rows)
```

Figure 52. Storage statistics of the compressed tables.

```
iot_assignment=# SELECT 'energy_readings' AS tbl, pg_size_pretty(table_bytes) AS data, pg_size_pretty(index_bytes) AS idx,
iot_assignment-#        pg_size_pretty(toast_bytes) AS toast, pg_size_pretty(total_bytes) AS total, total_bytes
iot_assignment-# FROM hypertable_detailed_size('energy_readings')
iot_assignment-# UNION ALL SELECT 'energy_time_compressed', pg_size_pretty(table_bytes), pg_size_pretty(index_bytes),
iot_assignment-#        pg_size_pretty(toast_bytes), pg_size_pretty(total_bytes), total_bytes FROM hypertable_detailed_size('energy_time_compressed')
iot_assignment-# UNION ALL SELECT 'energy_meter_compressed', pg_size_pretty(table_bytes), pg_size_pretty(index_bytes),
iot_assignment-#        pg_size_pretty(toast_bytes), pg_size_pretty(total_bytes), total_bytes FROM hypertable_detailed_size('energy_meter_compressed');
          tbl           |  data  |   idx   |  toast  |  total  | total_bytes
------------------------+--------+---------+---------+---------+-------------
 energy_readings        | 951 MB | 406 MB  | 0 bytes | 1357 MB |  1422598144
 energy_time_compressed | 15 MB  | 944 kB  | 423 MB  | 439 MB  |   459980800
 energy_meter_compressed | 27 MB  | 5584 kB | 469 MB  | 502 MB  |   525918208
(3 rows)
```

Figure 53. Storage statistics (second view, including the baseline).

4.3 Repeat the query comparison

benchmark.py --compressed repeated Q1–Q5 on both compressed tables with the 3.2 correctness checks and timing procedure (1 warm-up + 5 measured, warm-cache). The baseline was re-measured in the same session (22:09:56 CAT on 2026-10-12) so the comparison does not mix sessions. Improvement % = 100 × (baseline median − compressed median) / baseline median; positive = faster.

Query	Baseline 1 day (ms)	Time-only compressed (ms)	Improvement	Meter-segmented (ms)	Improvement
Q1	212.13	332.78	-56.9 %	1,217.43	-473.9 %
Q2	448.41	296.35	+33.9 %	168.73	+62.4 %
Q3	2,324.23	862.35	+62.9 %	412.11	+82.3 %
Q4	5.28	98.11	-1758.1 %	5.33	-0.9 %
Q5	1,005.66	583.86	+41.9 %	664.19	+34.0 %

Medians over 5 measured warm-cache runs; five-table view: the 3 h and week medians from 3.2 are 258.0/291.2 (Q1), 301.6/430.3 (Q2), 3,372.5/1,371.3 (Q3), 8.03/3.02 (Q4) and 1,089.7/1,005.3 (Q5) ms. Minimum and maximum per run are in results/group_machine_benchmark_4_3.csv.

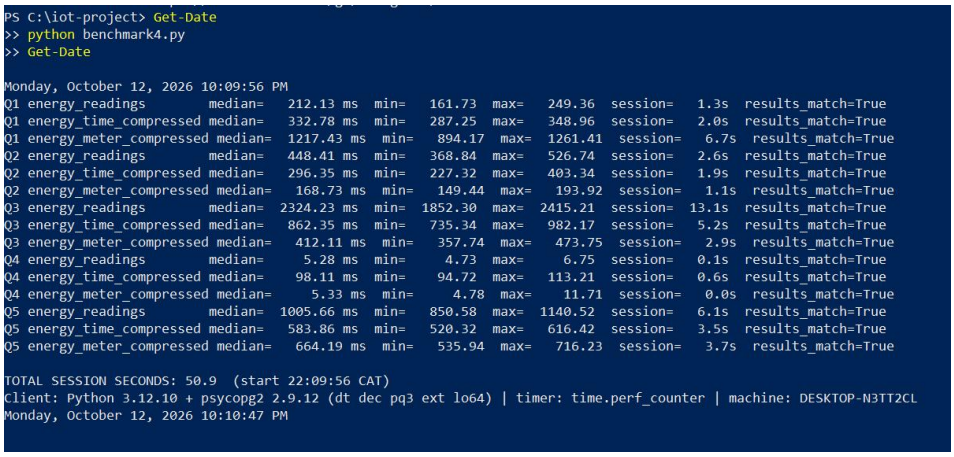


Figure 54. Completed 4.3 comparison on the five configurations.

4.4 Select a storage configuration

Effect of meter segmentation. For the single-meter workload Q4 the segmented table (5.33 ms) is as fast as the uncompressed baseline (5.28 ms, −0.9 %) because the meter is read directly from its own segments, while the time-only table is 98.11 ms (−1,758 %): with no segmentation every compressed batch mixes all 4,000 meters, so the meter's rows must be decompressed out of all batches. For workloads across all meters the segmented table is better on the grouped/filtered queries Q2 (+62.4 % vs +33.9 %) and Q3 (+82.3 % vs +62.9 %), but worse on Q1 (−473.9 % vs −56.9 %), and the time-only table wins Q5 (+41.9 % vs +34.0 %). Segmentation therefore helps queries aligned with meter_id, and not queries that aggregate over time across all meters.

Recommendation.

Use **1-day chunks with meter-segmented compression** (segment_by = meter_id, order_by = event_time DESC). Measured: it keeps 63.0 % of storage saved (1,357 MB → 502 MB, ratio 2.70), keeps single-meter lookup at baseline speed (Q4 5.33 ms vs 5.28 ms), and makes full-period per-meter totals 82.3 % faster (Q3 2,324 → 412 ms) and the top-ten-interval query 62.4 % faster (Q2).

5 Investigate the utility data

5.1 Identify demand patterns

Analysis_section5.py (SQL in sql/05_analysis.sql)

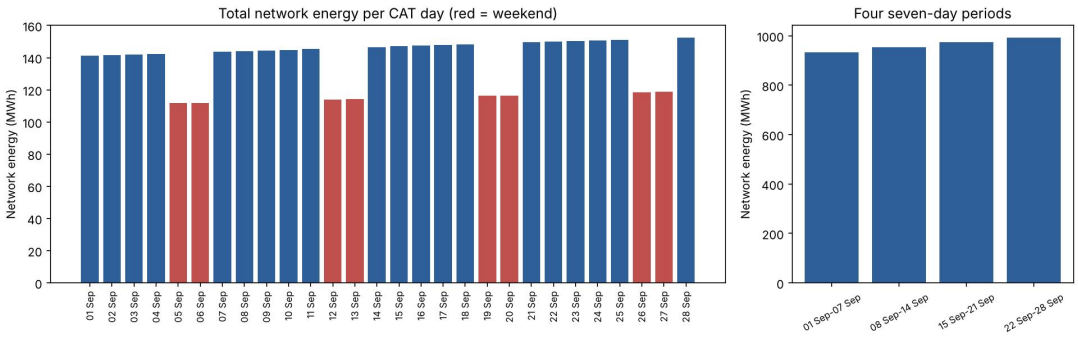
sums energy_kwh per CAT day (time_bucket('1 day', event_time, 'Africa/Kigali')) and per seven-day period;

network demand per interval is the sum of power_kw across meters at each event_time.

Seven-day period	Network energy (kWh)	Change vs previous
2026-09-01 – 2026-09-07	934,199.06	—
2026-09-08 – 2026-09-14	953,714.97	+2.09 %
2026-09-15 – 2026-09-21	973,230.03	+2.05 %
2026-09-22 – 2026-09-28	992,560.29	+1.99 %

Figure 55.

Rank	15-minute interval (CAT)	Network demand (kW)
1	2026-09-28 13:00	7,993.69
2	2026-09-28 12:45	7,943.95
3	2026-09-28 13:15	7,943.64



Total network energy per CAT day (kWh) and the four seven-day totals.

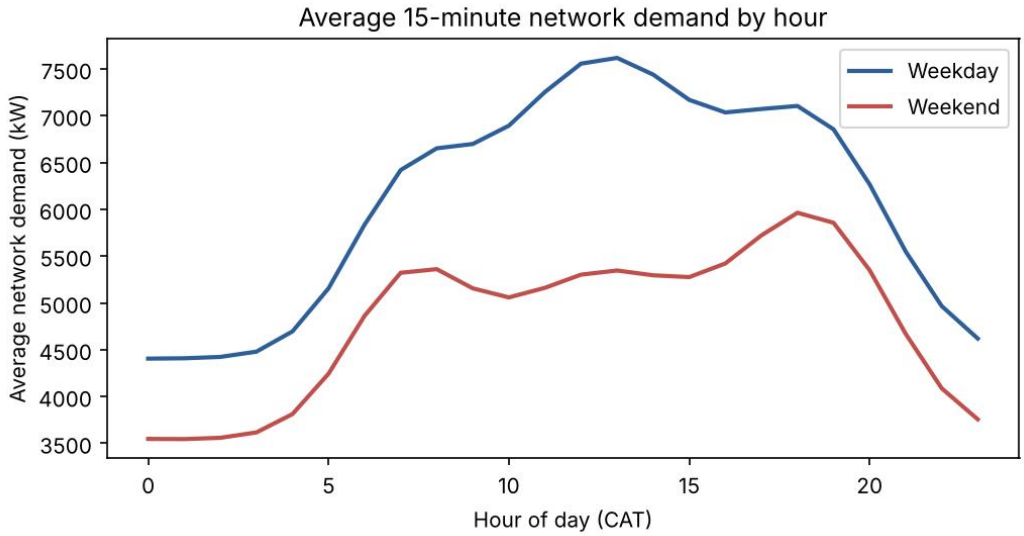


Figure 56. Average network demand (kW) by CAT hour, weekdays versus weekends.

```
smart_energy_grid=# SELECT
smart_energy_grid-#     (event_time AT TIME ZONE 'Africa/Kigali')::date AS cat_date,
smart_energy_grid-#     ROUND(SUM(energy_kwh)::numeric, 2) AS total_energy_kwh
smart_energy_grid-# FROM energy_readings
smart_energy_grid-# WHERE event_time >= '2026-09-01 00:00:00+02'
smart_energy_grid-#   AND event_time <  '2026-09-29 00:00:00+02'
smart_energy_grid-# GROUP BY 1
smart_energy_grid-# ORDER BY 1;
  cat_date  | total_energy_kwh
------------+------------------
 2026-09-01 |        141195.24
 2026-09-02 |        141533.90
 2026-09-03 |        141938.49
 2026-09-04 |        142301.48
 2026-09-05 |        111680.48
 2026-09-06 |        111950.74
 2026-09-07 |        143598.74
 2026-09-08 |        144127.00
 2026-09-09 |        144519.80
 2026-09-10 |        144818.08
 2026-09-11 |        145348.89
 2026-09-12 |        114025.49
 2026-09-13 |        114264.09
 2026-09-14 |        146611.63
 2026-09-15 |        147144.62
 2026-09-16 |        147508.12
 2026-09-17 |        147968.92
 2026-09-18 |        148320.20
 2026-09-19 |        116242.45
 2026-09-20 |        116493.00
 2026-09-21 |        149552.72
 2026-09-22 |        150056.53
 2026-09-23 |        150430.65
 2026-09-24 |        150800.61
```

Figure 57. : executed analysis: total energy per CAT day.

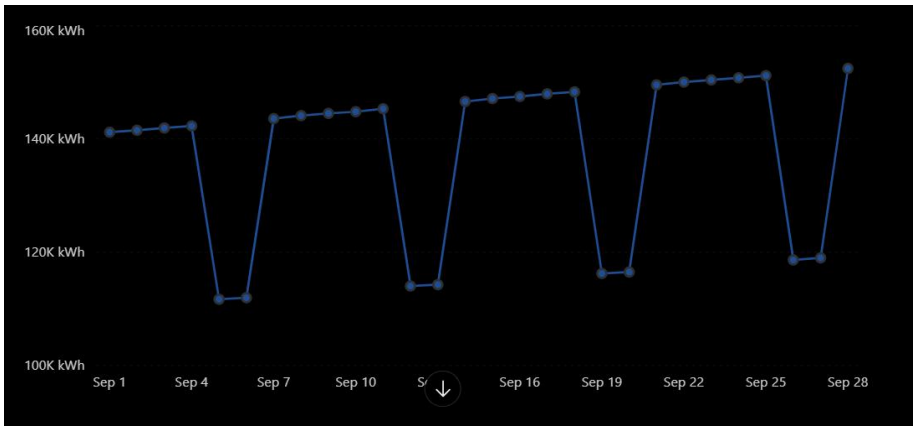


Figure 58. : daily network energy chart.

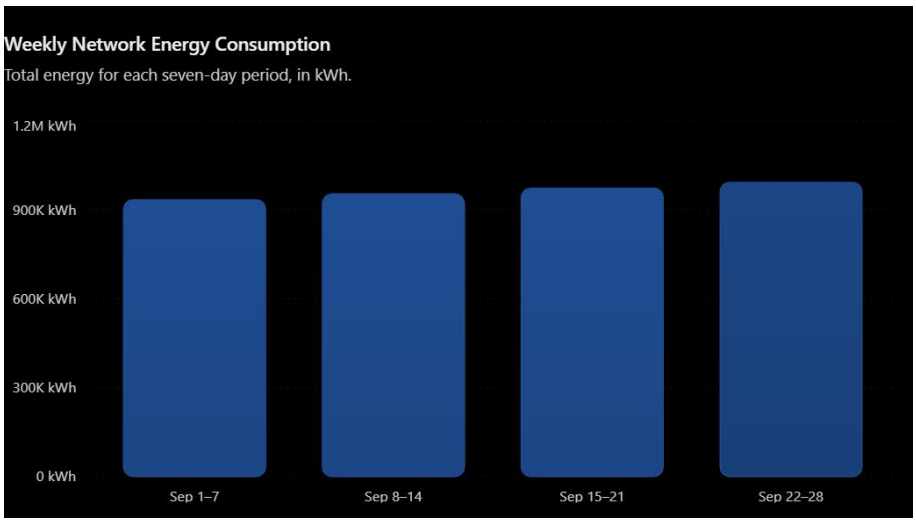


Figure 59. : network energy for each seven-day period.

Highest recorded demand · 28 September 2026

7,993.69 kW

13:00 CAT (1:00 PM)

Rank	Time (CAT)	Demand (kW)
1	13:00	7,993.69
2	12:45	7,943.95
3	13:15	7,943.64

All three peaks occurred on 28 September, within a 30-minute period. The SQL query reported 3,965, 3,969 and 3,952 meters, respectively.

Figure 60. : the three highest 15-minute demand intervals .

Patterns.

- Weekly energy rises steadily: 934.2 → 953.7 → 973.2 → 992.6 thousand kWh (about +2 % per week, a trend of about +0.3 % per day).
- Saturdays and Sundays use clearly less (about 112–119 thousand kWh per day against 141–152 thousand on weekdays, roughly 20 % lower).
- On weekdays demand peaks at 13:00 (7,618 kW average); at weekends it peaks in the evening (18:00, 5,963 kW).
- The three highest 15-minute intervals are all on Monday 28 September around midday (12:45–13:15, up to 7,993.7 kW), i.e. at the end of the growth trend.

The highest daily total was 152,464.16 kWh on Monday 28 September and the lowest 111,680.48 kWh on Saturday 5 September; at the three peak intervals 3,965, 3,969 and 3,952 meters reported. From the first to the fourth week energy rose 6.2 %. **Operational implications.** Capacity, generation scheduling and maintenance windows should follow the weekday midday peak; weekends and the night trough (about 4,400 kW) are the low-risk periods for planned work. Growth of about 2 % per week means peak headroom shrinks week by week and should be reviewed.

kW versus kWh. kW is power, an instantaneous rate; the peak intervals above are demand the network must be able to supply at that moment. kWh is energy, power accumulated over time (kW × hours; a 15-minute reading uses ≈ power_kw × 0.25 kWh), the quantity customers are billed for. A short high-kW peak can matter for capacity although it adds little kWh, while a modest, long, flat load can add many kWh without any peak.

5.2 Compare regions and customer types

Readings are joined to meters on meter_id; per-meter value = group energy ÷ registered meters in the group (all 28 days).

Region	Meters	Energy (kWh)	kWh per meter
Region 1	400	383,539.8	958.85
Region 2	400	378,448.9	946.12
Region 3	400	392,376.6	980.94
Region 4	400	324,404.5	811.01
Region 5	400	411,040.4	1,027.60
Region 6	400	346,512.8	866.28
Region 7	400	362,288.5	905.72
Region 8	400	503,475.8	1,258.69
Region 9	400	400,467.2	1,001.17
Region 10	400	351,149.8	877.87

Customer type	Meters	Energy (kWh)	kWh per meter
Commercial	1,010	825,475.2	817.30
Industrial	406	2,220,672.0	5,469.64
Residential	2,584	807,557.2	312.52

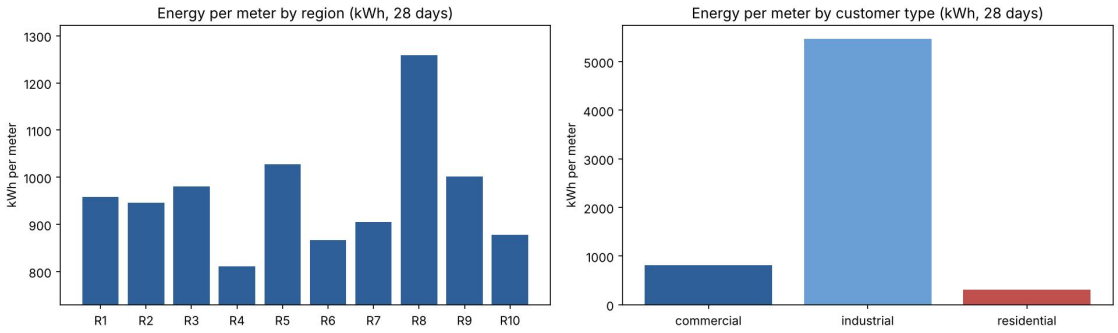


Figure 61. Energy per meter by region and by customer type.

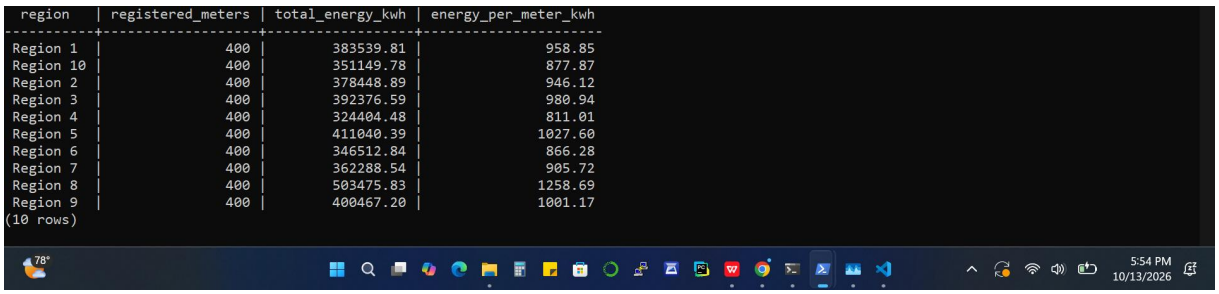


Figure 62. : totals, registered meters and kWh per meter by region.

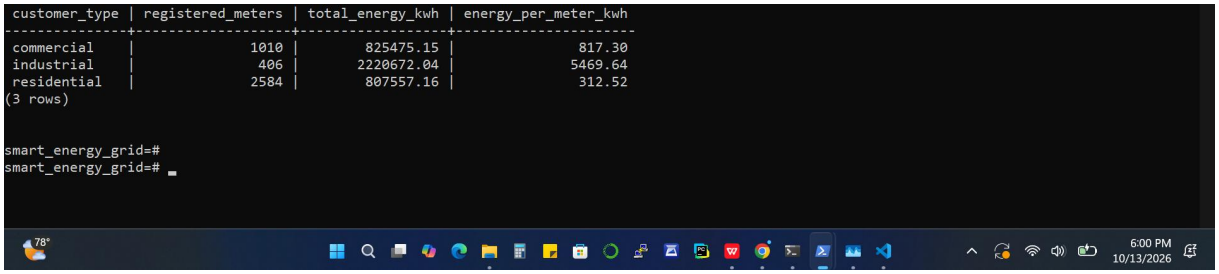


Figure 63. : totals, registered meters and kWh per meter by customer type.

What remains after normalization. Every region has 400 meters, so totals and per-meter values differ only in scale, yet per-meter energy still ranges from 811 kWh (Region 4) to 1,259 kWh (Region 8), a 55 % spread. Among customer types the difference is far larger: an industrial meter used 5,470 kWh, about 17.5× a residential meter (313 kWh) and 6.7× a commercial meter (817 kWh). The registered industrial meter counts differ by region (for example Region 8 has 61 industrial meters, Region 4 only 29), which explains most of the regional gap: regional differences reflect customer mix, not a different behaviour of the same customer type.

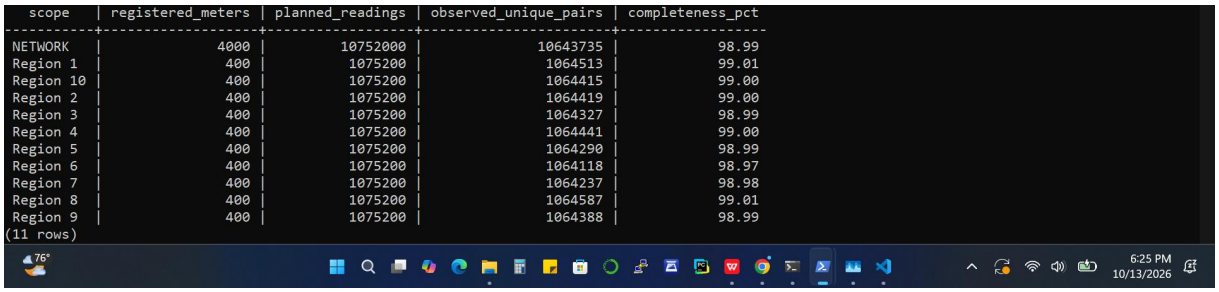
Implication. Regional comparisons should be made by customer type (or weighted by mix), and industrial customers deserve their own monitoring and tariffs: 406 meters (10 %) consume 57.6 % of all energy.

5.3 Assess reporting quality

Scope	Observed unique pairs	Planned (meters × 28 × 96)	Completeness (%)
Whole network	10,643,735	10,752,000	98.9931
Region 1	1,064,513	1,075,200	99.0060
Region 2	1,064,419	1,075,200	98.9973
Region 3	1,064,327	1,075,200	98.9887
Region 4	1,064,441	1,075,200	98.9993
Region 5	1,064,290	1,075,200	98.9853
Region 6	1,064,118	1,075,200	98.9693
Region 7	1,064,237	1,075,200	98.9804
Region 8	1,064,587	1,075,200	99.0129
Region 9	1,064,388	1,075,200	98.9944
Region 10	1,064,415	1,075,200	98.9969

Readings with event_time on 21 September	Rows	Energy (kWh)
Known at 00:00 on 22 September (received_time ≤ 00:00, 22 Sep)	379,505	149,400.545
All arrivals (final)	380,036	149,552.723
Difference	531	152.178 = 0.1018 % of the final total

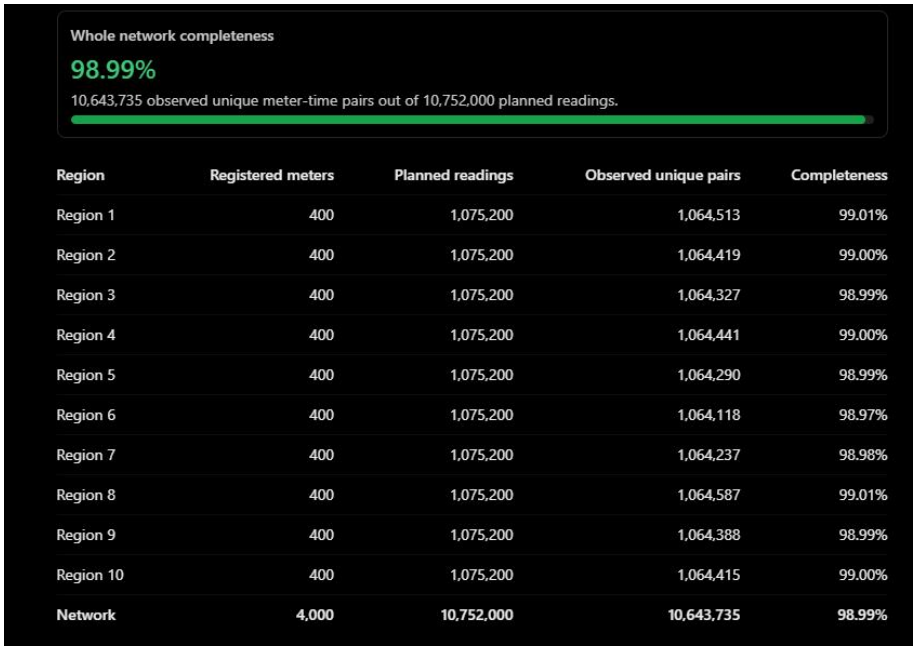
Effects. Regional completeness is uniform (98.969 % – 99.013 %), so missing readings are spread evenly and no region is systematically worse; about 1 % of planned readings are never received and their energy is simply absent from totals (understated, not zero). Delayed readings (2 % of readings arrive 30–180 minutes late) are missing from a report taken too early: the day-end report for 21 September understates the final total by 152.178 kWh (0.1018 %), and it is corrected only when the late readings arrive. For billing and day-ahead reports, totals should be labelled provisional until the 3-hour delay window has passed.



scope	registered_meters	planned_readings	observed_unique_pairs	completeness_pct
NETWORK	4000	10752000	10643735	98.99
Region 1	400	1075200	1064513	99.01
Region 10	400	1075200	1064415	99.00
Region 2	400	1075200	1064419	99.00
Region 3	400	1075200	1064327	98.99
Region 4	400	1075200	1064441	99.00
Region 5	400	1075200	1064290	98.99
Region 6	400	1075200	1064118	98.97
Region 7	400	1075200	1064237	98.98
Region 8	400	1075200	1064587	99.01
Region 9	400	1075200	1064388	98.99

(11 rows)

Figure 64. : completeness for the whole network and each region.



Whole network completeness

98.99%

10,643,735 observed unique meter-time pairs out of 10,752,000 planned readings.

Region	Registered meters	Planned readings	Observed unique pairs	Completeness
Region 1	400	1,075,200	1,064,513	99.01%
Region 2	400	1,075,200	1,064,419	99.00%
Region 3	400	1,075,200	1,064,327	98.99%
Region 4	400	1,075,200	1,064,441	99.00%
Region 5	400	1,075,200	1,064,290	98.99%
Region 6	400	1,075,200	1,064,118	98.97%
Region 7	400	1,075,200	1,064,237	98.98%
Region 8	400	1,075,200	1,064,587	99.01%
Region 9	400	1,075,200	1,064,388	98.99%
Region 10	400	1,075,200	1,064,415	99.00%
Network	**4,000**	**10,752,000**	**10,643,735**	**98.99%**

Figure 65. : network completeness 98.99 % (10,643,735 of 10,752,000) with the regional table.

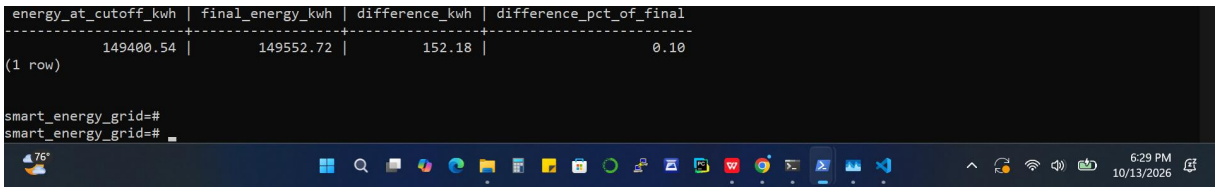


Figure 66. : 21 September total known at 00:00 on 22 September (149,400.54 kWh) versus final (149,552.72 kWh): difference 152.18 kWh = 0.10 %.

5.4 Investigate abnormal consumption

Rule (defined before the labels are used). For every meter, the expected power of a reading is the median power of the same meter at the same time of day on the same kind of day (weekday or weekend) over the 28 days. ratio = power_kw / expected. A reading is flagged as a **spike** if ratio ≥ 2.0 and as an **unusual drop** if ratio ≤ 0.25; each flag carries a meter_id and event_time. Reason for the thresholds: normal readings vary by about ±15 % plus a slow upward trend (ratio percentiles: 0.1 % = 0.789, 1 % = 0.822, median 1.000, 99 % = 1.199, 99.9 % = 1.250), so 2.0 and 0.25 lie far outside normal variation; the median is robust because a 3-hour event changes only one of the 20 weekday (8 weekend) samples that define the expectation.

Measure	Result
Flagged readings (meter_id, event_time)	474
Injected events detected / missed (of 40)	40 / 0
Detected by type	20 of 20 demand spikes, 20 of 20 unusual drops
False flags (flags outside every injected interval of that meter)	0
Observed readings outside injected intervals	10,643,261
False flags as % of readings outside injected intervals	0.0000 %
Meters with at least one false flag / 4,000 (percentage of meters affected by false flags)	0 / 4,000 = 0.0 %

Sensitivity of the thresholds (same data, other thresholds):

Spike ≥	Drop ≤	Flags	Detected	False flags	False % of outside	Meters affected
1.1	0.9	3,119,219	40/40	3,118,745	29.3025 %	100.0 %
1.2	0.8	127,598	40/40	127,124	1.1944 %	100.0 %
1.3	0.75	1,151	40/40	677	0.0064 %	15.5 %
1.4	0.7	474	40/40	0	0.0000 %	0.0 %
1.5	0.5	474	40/40	0	0.0000 %	0.0 %
1.7	0.4	474	40/40	0	0.0000 %	0.0 %
2.0	0.25	474	40/40	0	0.0000 %	0.0 %
2.5	0.1	424	40/40	0	0.0000 %	0.0 %

Missed events and false alarms. No event was missed: the 474 flags cover every event (a 3-hour event has 12 readings; a few readings of an event can stay unflagged where the injected change is smaller than the threshold or the reading is missing). There were no false alarms with the chosen rule. With thresholds of 1.3 / 0.75 or looser, detection stays at 40/40 but normal noise creates hundreds to millions of false flags and affects 15.5 % to 100 % of meters; at 1.1/0.9 the rule flags 29 % of normal readings. Stricter thresholds (2.5 / 0.1) give fewer flags (424 instead of 474), although all 40 events are still found. The same-data caveat applies: the generator's noise is simple; real meters (holidays, solar, equipment changes) would raise the false-alarm rate.

5.5 Management recommendation

- **Operations.** Plan for the weekday midday peak (13:00, up to 7,993.7 kW on Monday 28 September) and let demand growth of about 2 % per week trigger a capacity review; schedule planned work at night or at weekends, when demand is 20–40 % lower. Treat industrial customers (10 % of meters, 57.6 % of energy) as a separate monitoring group, and compare regions only after allowing for customer mix. About 1 % of readings never arrive and 2 % arrive up to three hours late, so label same-day totals provisional (21 September was 0.10 % low, 152 kWh, at midnight) and keep gaps as gaps rather than zeros. The simple rule of 2.0× / 0.25× the meter's own typical value found all 40 injected abnormal events with no false flags and can run as an alert.

- **Storage and reporting.** Keep one-day chunks and compress them with segmentby = meter_id: it cut storage from 1,357 MB to 502 MB (ratio 2.70) while keeping single-meter queries at baseline speed (5.33 ms against 5.28 ms) and making per-meter totals 82 % faster (2,324 to 412 ms). Serve the dashboards from continuous aggregates: they matched the raw numbers on every panel and refreshed the dashboard several times faster in our timing (Section 7.4) for a small amount of extra storage.
- **Forecast.** Use the model for day-ahead regional planning. On the 1,680 test region-hours its mean absolute error was 4.55 kWh against 42.71 kWh for the previous-day baseline (−89 %), about 0.8 % of the average hourly consumption (590.8 kWh); the largest gains were on days after a weekend change (Saturday 26 and Monday 28 September). Re-train regularly, continue to use the baseline as a safety check, and add a holiday calendar before relying on it for special days.

6 Create and investigate continuous aggregates

6.1 Create the three aggregate views

sql/06_caggs.sql creates three continuous aggregates over energy_readings, all with WITH NO DATA and timescaledb.materialized_only = true (real-time aggregation off): cagg_15min_region (15-minute bucket × region), cagg_hour_region (CAT hour × region) and cagg_day_meter (CAT day × meter_id). The regional views use an inner join with meters on meter_id. Each stores bucket, grouping key, sum_energy_kwh, sum_power_kw and reading_count.

```
CREATE MATERIALIZED VIEW cagg_hour_region
WITH (timescaledb.continuous, timescaledb.materialized_only = true) AS
SELECT time_bucket('1 hour', r.event_time, 'Africa/Kigali') AS bucket, m.region AS region,
       sum(r.energy_kwh) AS sum_energy_kwh, sum(r.power_kw) AS sum_power_kw, count(*)
AS reading_count
FROM energy_readings r JOIN meters m ON m.meter_id = r.meter_id
GROUP BY 1, 2 WITH NO DATA;
-- cagg_15min_region: time_bucket('15 minutes', event_time), region (same join)
-- cagg_day_meter:    time_bucket('1 day', event_time, 'Africa/Kigali'), meter_id (no join)
```

caggs_refresh.py refreshes the entire generated period manually (refresh_continuous_aggregate(view, '2026-09-01', '2026-09-29')) and measures duration and stored size. Only afterwards the policies were added (sql/06b_cagg_policies.sql): start_offset 3 days, end_offset 1 hour, schedule every 15 minutes. Because these offsets are relative to the clock of the database (October 2026), the policies do not refresh the September data.

View	Rows	Readings covered	Refresh (s)	Data	Index	Total
cagg_15min_region	26,880	10,643,735	7.9	2.2 MB	2.1 MB	4.3 MB
cagg_hour_region	6,720	10,643,735	16.4	0.6 MB	0.6 MB	1.3 MB
cagg_day_meter	112,000	10,643,735	12.3	7.8 MB	5.4 MB	13.2 MB
All three			36.6			18.8 MB

Validation-run values. Rows, readings covered (10,643,735 in each view) and the first and last bucket are deterministic. Refresh seconds and sizes above were measured on the validation machine; replace them with the values printed on the group machine by python caggs_refresh.py.

6.2 Test freshness and delayed arrivals

freshness_test.py (SQL in sql/06c_freshness_test.sql) creates the separate test hypertable fresh_test (event_time TIMESTAMPTZ, energy_kwh DOUBLE PRECISION) and the hourly aggregate fresh_test_hourly (sum of energy and row count) with WITH NO DATA, real-time aggregation off and no refresh policy. It records the CAT test time, H (start of the most recent completed CAT hour) and H − 48 hours, and inserts relative to them.

Validation-run timeline (replace with the group's run). Test time 2026-10-13 19:03:10 CAT; H = 2026-10-13 18:00; older hour (H − 48 h) = 2026-10-11 18:00; inserts at 2026-10-11 18:10 (1 kWh), 2026-10-11 18:25 (0.5 kWh) and 2026-10-13 18:10 (2 kWh). Policy window of 6.1 at that moment: 2026-10-10 19:03 to 2026-10-13 18:03.

Stage	Older hour: raw	Older hour: aggregate	Hour H: raw	Hour H: aggregate
1. Initial older-hour refresh, real-time off	1 kWh, 1 row(s)	1 kWh, 1 row(s)	no rows	no bucket returned
2. After both new inserts, before refresh, real-time off	1.5 kWh, 2 row(s)	1 kWh, 1 row(s)	2 kWh, 1 row(s)	no bucket returned
3. Real-time on, still before refresh	1.5 kWh, 2 row(s)	1 kWh, 1 row(s)	2 kWh, 1 row(s)	2 kWh, 1 row(s)
4. After manual refresh of both hours	1.5 kWh, 2 row(s)	1.5 kWh, 2 row(s)	2 kWh, 1 row(s)	2 kWh, 1 row(s)

Explanation. (1) After the first manual refresh of the older hour the aggregate holds that single reading (1 kWh, 1 row) and hour H has no bucket at all: absent, not zero. (2) The two later inserts exist in the raw table (older hour 1.5 kWh, 2 rows; H 2 kWh, 1 row), but the aggregate is a stored copy: with real-time aggregation off it still shows 1 kWh/1 row and no bucket for H, so a late reading in an already materialized hour is invisible until that hour is refreshed. (3) With real-time aggregation on, the view adds the not-yet-materialized newest data: hour H now appears (2 kWh, 1 row), but the older hour is unchanged (1 kWh, 1 row) because it lies inside the materialized region; real-time aggregation shows new data, not changes to old data. (4) After the manual refresh of both hours the aggregate equals the raw table (1.5 kWh, 2 rows; 2 kWh, 1 row). At the recorded test time the 6.1 policy window starts 3 days before and ends 1 hour before the clock, so it could refresh the older hour (its whole hour lies between test time − 3 days and test time − 1 hour) but not hour H: the window ends 1 hour before the test time, a point that falls inside hour H, so H is never completely inside the refreshed range and only a manual refresh makes it visible. (The 6.2 timeline in the assignment gives the same results.)

7 Build and compare dashboards

7.1 Build the Raw dashboard

The dashboards are written with Plotly Dash (dashboard/app.py, queries in dashboard/queries.py, run as python dashboard/app.py --mode raw --port 8050). The same code serves both dashboards; the mode selects the query set, the title ("Raw" or "Aggregate"), the layout, the filters (start and end CAT day, selected meter) and the five panels: P1 hourly regional energy (kWh), P2 daily network energy (kWh), P3 network energy per seven-day period from 1, 8, 15, 22 September (kWh), P4 daily energy of the selected meter (kWh), P5 completeness by region (%). Every panel is limited to the selected range; the page shows each panel's query time and the run number. Dash does not cache query results (every refresh sends all five queries). **Raw** queries energy_readings directly,

```
-- P1 (Raw)
SELECT time_bucket('1 hour', r.event_time, 'Africa/Kigali') AS bucket, m.region, sum(r.energy_kwh) AS energy_kwh
FROM energy_readings r JOIN meters m ON m.meter_id = r.meter_id
WHERE r.event_time >= :t0 AND r.event_time < :t1 GROUP BY 1, 2 ORDER BY 1, 2;
-- P2: time_bucket('1 day', event_time, 'Africa/Kigali'), sum(energy_kwh)  P4: same plus WHERE meter_id = :meter
-- P3: 1 Sep + 7*((cat_day - 1 Sep)/7) as period_start, sum(energy_kwh)
-- P5: count(*) per region (join meters) / (400 meters x days x 96) * 100
```

joining meters where regions are needed:

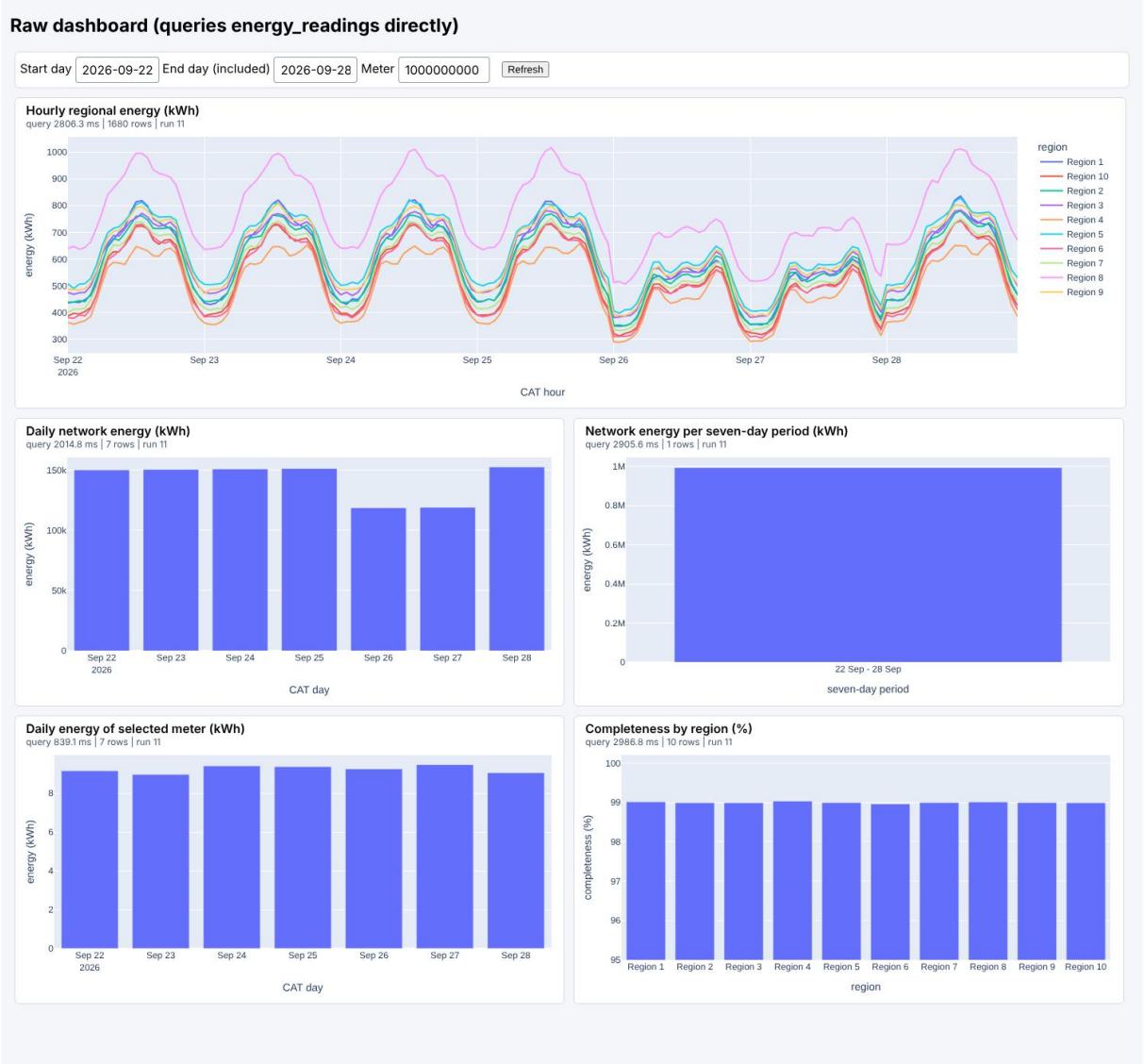


Figure 67. Raw dashboard, 22–28 September, meter 1000000000 (validation-run screenshot; the group screenshot replaces it).

7.2 Build the Aggregate dashboard

Aggregate (--mode aggregate --port 8051) has the same five panels, layout, filters and date range, but its measurement queries read only the continuous aggregates: P1 from cagg_hour_region, P2 and P3 from cagg_15min_region (re-bucketed by CAT day or seven-day period), P4 from cagg_day_meter, P5 observed counts from cagg_15min_region. Stored energy sums and reading counts are summed when buckets are combined; planned readings are computed from the registered meter count per region (meters) times the selected days times 96.

```
-- P1 (Aggregate)
SELECT bucket, region, sum(sum_energy_kwh) AS energy_kwh, sum(reading_count) AS readings
FROM cagg_hour_region WHERE bucket >= :t0 AND bucket < :t1 GROUP BY 1, 2 ORDER
```

```
BY 1, 2;
-- P2/P3: sum(sum_energy_kwh) from cagg_15min_region grouped by time_bucket('1 day', bucket, 'Africa/Kigali') / seven-day period
-- P4: SELECT bucket, sum_energy_kwh FROM cagg_day_meter WHERE meter_id = :meter AND bucket >= :t0 AND bucket < :t1
-- P5: sum(reading_count) per region from cagg_15min_region; planned = meters x days x 96 (meters table)
```



Figure 68. Aggregate dashboard with the same filters

7.3 Verify matching results

Both dashboards were set to 22 September 00:00 through 29 September 00:00 (exclusive), all regions, meter 1000000000. python bench_dashboards.py verify queries the numerical data behind every panel in both modes, matches rows by bucket, region or meter, and compares keys, reading counts, energy (limit 0.001 kWh) and completeness (limit 0.000001 percentage points). The exports are in results/dashboard_export/.

Panel	Rows compared	Identical keys	Max | energy| (kWh)	Max |count|	Max | completeness| (pp)	Result
P1 Hourly regional energy (kWh)	1680	yes	3.52e-12	0	0.0e+00	PASS
P2 Daily network energy (kWh)	7	yes	5.06e-09	0	0.0e+00	PASS
P3 Network energy per seven-day period (kWh)	1	yes	1.01e-08	0	0.0e+00	PASS
P4 Daily energy of selected meter (kWh)	7	yes	1.78e-15	0	0.0e+00	PASS
P5 Completeness by region (%)	10	yes	0.00e+00	0	0.0e+00	PASS

All five panels match: identical keys, identical reading counts, energy differences far below 0.001 kWh (floating-point summation order) and no completeness difference. No correction was needed before timing.

7.4 Measure performance and judge the tradeoff

Method: python bench_dashboards.py time pauses the refresh and compression policy jobs, starts each dashboard as its own process, and uses one headless Chromium browser (Playwright) and the same machine and database connection for both. It presses Refresh once as warm-up and ten more times as measured runs. Per-panel query duration = time.perf_counter() around the panel query inside the app; whole-dashboard time = performance.now() in the page from the click until all five panels display the new run (plus two animation frames). Result caching is off. Jobs are resumed afterwards.

Panel	Raw median	min	max	Aggregate median	min	max	Speed-up
P1 hourly regional energy	2,579.5	2,059.1	2,935.8	18.9	8.4	62.2	136.8×
P2 daily network energy	2,038.4	1,700.1	2,414.3	24.6	3.5	49.1	83.0×
P3 seven-day periods	2,905.0	2,287.3	3,213.2	20.6	13.7	135.7	141.4×
P4 selected meter	826.4	394.2	955.4	3.9	0.7	40.1	214.6×
P5 completeness	2,714.7	1,437.0	3,104.1	36.4	14.0	118.6	74.6×
Whole dashboard	3,025.1	2,943.4	3,412.9	423.9	344.4	451.3	7.1×

All values in milliseconds, 10 measured refreshes. **Validation-run measurements** (2-vCPU Linux machine): the group's own results from bench_dashboards.py time replace this table.

- **Cost of the aggregates (from 6.1, validation run):** 18.8 MB of extra storage (about 1.3 % of the 1,357 MB uncompressed table) and an initial refresh of 36.6 s for the three views; afterwards only recent buckets are refreshed (every 15 minutes).

- **Judgement.** The whole dashboard refreshes 7.1× faster (3,025 ms → 424 ms), and the panels' queries are about 75–215× faster (for example P4 826 → 3.9 ms). The remaining 424 ms in Aggregate mode is page rendering, not database time. The extra cost is small: about 19 MB of storage and a one-off refresh of about 37 s, plus a 15-minute refresh job. For a dashboard that many operators open repeatedly, the saving of about 2.6 s per refresh justifies this cost. Two caveats: the aggregates are only as fresh as their last refresh (6.2) and only answer the questions they were designed for (hour × region, 15-minute × region, day × meter).

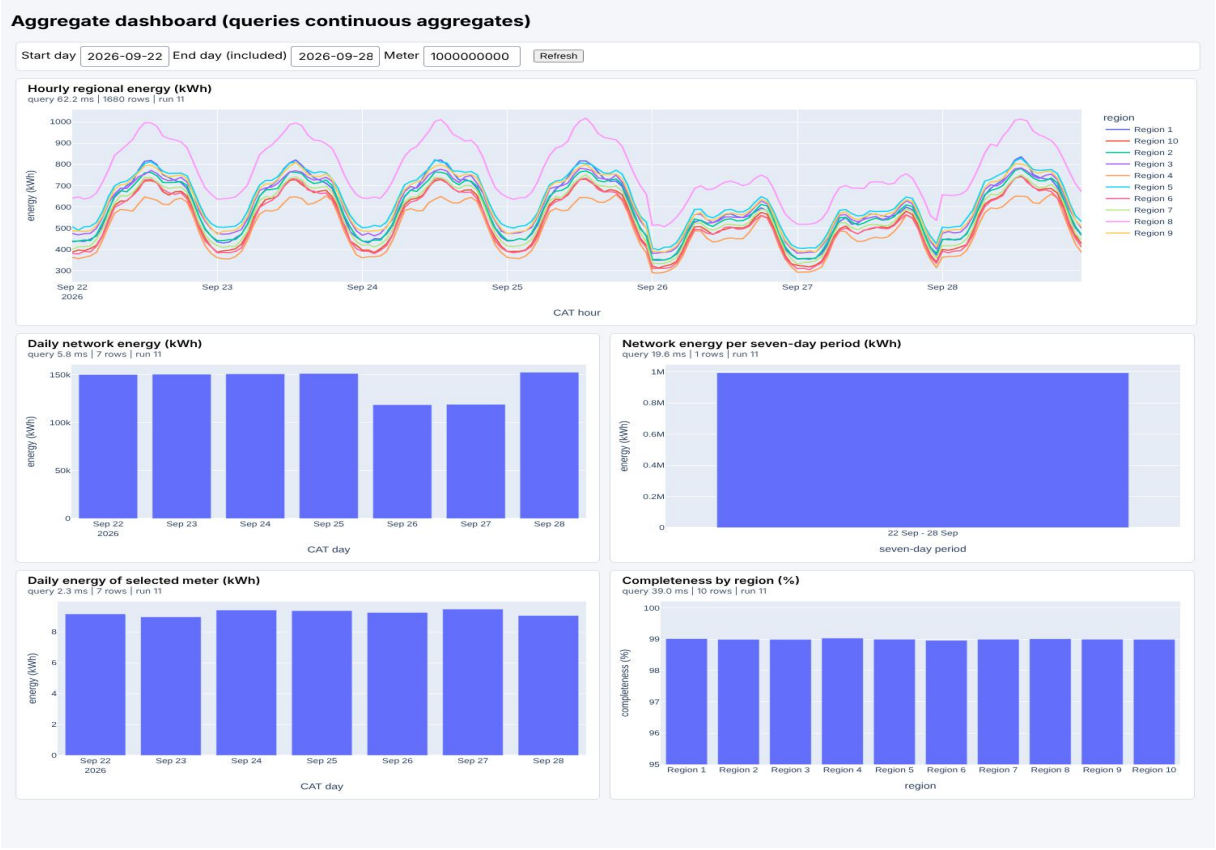


Figure 69. Final Aggregate dashboard screenshot used during the timing run (validation run).

8 Forecast the next day

Target: hourly regional consumption (kWh) per CAT hour, summed from cleaned energy_readings joined to meters (s8_forecast.py). This is an offline forecast of observed consumption; completed historical hourly totals are treated as final and the remaining missing readings are discussed as a limitation.

8.1 Prepare chronological data and features

Dataset. One row per region and CAT hour: 10 regions × 672 hours = **6,720 rows**, none absent (0 absent region-hours). Each row holds the sum of energy_kwh (the observed target) and the reading count: minimum 1,562, median 1,584, maximum 1,597 readings per region-hour (planned 1,600 = 400 meters × 4), on average 98.993 % of the planned readings. Observed totals are kept as the evaluation targets: no imputation is applied to targets, and missing readings simply make an hourly total slightly lower. Missing predictor values arise only at the start of the data (before lag sources exist) and those rows are excluded rather than imputed.

Split	Target dates (CAT)	Rows	First hour	Last hour
Training	1–14 September	3,360	2026-09-01 00:00	2026-09-14 23:00
Validation	15–21 September	1,680	2026-09-15 00:00	2026-09-21 23:00
Test	22–28 September	1,680	2026-09-22 00:00	2026-09-28 23:00

Forecast origin and features. For every target hour the forecast origin is 00:00 CAT at the start of its day, so only information from before that midnight may be used. Preprocessing

(a StandardScaler for the numeric features and one-hot encoding of region) is fitted on the retained training rows only.

Feature	Lag / window (hours)	Earliest usable target timestamp
lag_24h	24	2026-09-02 00:00
lag_48h	48	2026-09-03 00:00
lag_168h	168	2026-09-08 00:00
prev_day_mean_24h	24	2026-09-02 00:00
hour_of_day	calendar	2026-09-01 00:00
is_weekend	calendar	2026-09-01 00:00
region	calendar	2026-09-01 00:00

The chosen initial look back is **168 hours** (the longest lag, lag_168h), so the earliest usable target is 2026-09-08 00:00. Training rows before exclusions: **3,360**; excluded for lacking the look back: **1,680**; retained: **1,680**. The same retained rows are used for every model configuration. lag_24h (previous day, same hour), lag_48h, lag_168h (same hour one week earlier) and prev_day_mean_24h (mean of the previous day) lie wholly before the forecast origin; no feature uses a preceding hour of the forecast day.

Leakage check. For all 5,040 usable rows the program asserts that the source timestamps of lag_24h, lag_48h, lag_168h and the end of the previous-day window are strictly earlier than the 00:00 forecast origin of the target's day: the check passed.

8.2 Evaluate the previous-day baseline

At 00:00 CAT of each test day the baseline forecasts every one of the 24 hours of the day with the previous day's value of the same region and hour, for all 10 regions: 7 days × 24 hours × 10 regions = **1,680 predictions**. Test-set metrics: **MAE = 42.708 kWh, RMSE = 79.403 kWh** (mean actual hourly consumption 590.8 kWh).

Region	Hour (CAT)	Actual (kWh)	Baseline prediction (kWh)	Error (pred − actual)
Region 1	2026-09-28 13:00	835.61	565.27	-270.34
Region 2	2026-09-28 13:00	780.13	546.91	-233.21
Region 3	2026-09-28 13:00	783.00	554.69	-228.32
Region 4	2026-09-28 18:00	659.90	561.41	-98.50
Region 5	2026-09-28 13:00	828.23	577.36	-250.87

(Sample rows: the baseline's forecasts at the peak hours of Regions 1–5; the complete 1,680 predictions are in results/forecast_predictions.csv.)

8.3 Train and evaluate one regression model

Model: Ridge regression (scikit-learn) with five configurations that differ only in the regularization strength alpha. **Shared predictors:** lag_24h, lag_48h, lag_168h, prev_day_mean_24h, is_weekend, hour_of_day and region (one-hot). Numeric predictors are standardized; the scaler and encoder are fitted on the training split only. Each configuration is trained on the 1,680 retained training rows and evaluated on the 1,680 validation rows.

Config	Model settings	Training rows	Validation rows	Validation MAE (kWh)	Validation RMSE (kWh)
R1	Ridge, alpha = 0.01	1,680	1,680	4.454	5.660
R2	Ridge, alpha = 0.1	1,680	1,680	4.455	5.660
R3	Ridge, alpha = 1.0	1,680	1,680	4.466	5.675
R4	Ridge, alpha = 10.0	1,680	1,680	4.830	6.181
R5	Ridge, alpha = 100.0	1,680	1,680	9.348	13.397

Previous-day baseline on the validation split: MAE 42.053 kWh, RMSE 78.337 kWh. **Selected: R1 (alpha = 0.01), lowest validation MAE 4.454 kWh.** The fitted model and preprocessing were frozen (saved as results/frozen_model.joblib) before the test split was touched. The test predictions use only information available before each daily origin: lag values from earlier completed test days

may be used as inputs, but the model and scaler are not refitted. Number of test predictions: 1,680 (7 days × 24 hours × 10 regions). Test MAE 4.555 kWh, RMSE 5.786 kWh.

8.4 Judge forecast usefulness

Scope (1,680 test observations)	Baseline MAE	Model MAE	Baseline RMSE	Model RMSE
All regions	42.708	4.555	79.403	5.786
Region 1 (mean 588 kWh)	44.873	4.721	83.696	5.911
Region 2 (mean 581 kWh)	41.629	4.539	76.811	5.674
Region 3 (mean 601 kWh)	42.754	4.293	77.607	5.595
Region 4 (mean 498 kWh)	34.856	4.065	64.233	5.146
Region 5 (mean 630 kWh)	44.400	5.223	81.084	6.574
Region 6 (mean 531 kWh)	39.426	3.678	74.766	4.690
Region 7 (mean 556 kWh)	39.688	4.401	73.945	5.712
Region 8 (mean 772 kWh)	55.967	5.709	102.057	7.081
Region 9 (mean 614 kWh)	43.772	4.665	80.178	5.799
Region 10 (mean 538 kWh)	39.717	4.253	74.282	5.319

All values in kWh; error = predicted − actual; MAE = mean of |error|, RMSE = √mean(error²).

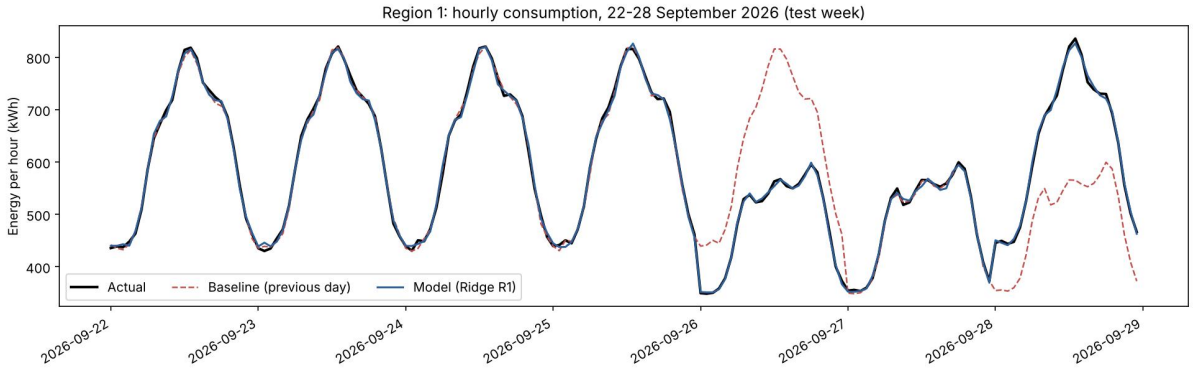


Figure 70. Actual, previous-day baseline and Ridge forecast for Region 1 over the seven test days.

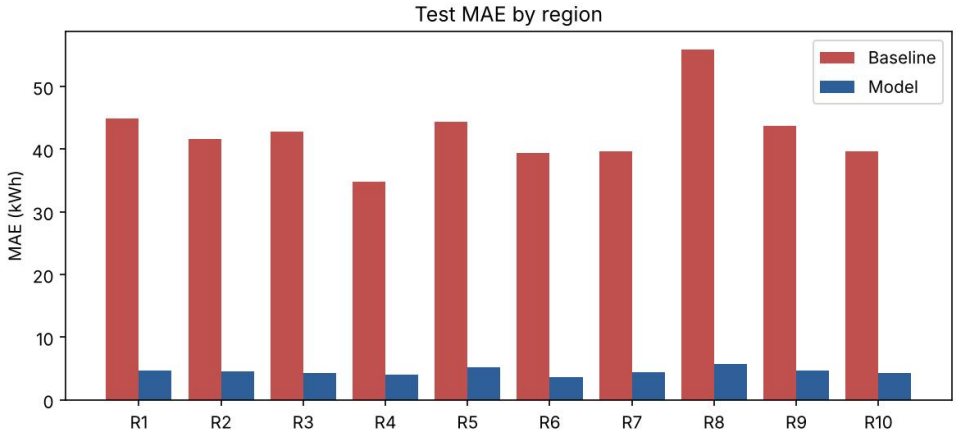


Figure 71. MAE by region: baseline versus model.

Highest actual test hour per region (ties broken by earliest time):

Region	Peak hour (CAT)	Actual	Baseline pred.	Baseline |err|	Model pred.	Model |err|
Region 1	2026-09-28 13:00	835.6	565.3	270.3	826.4	9.2
Region 2	2026-09-28 13:00	780.1	546.9	233.2	773.9	6.2
Region 3	2026-09-28 13:00	783.0	554.7	228.3	790.5	7.5
Region 4	2026-09-28 18:00	659.9	561.4	98.5	655.5	4.4
Region 5	2026-09-28 13:00	828.2	577.4	250.9	826.2	2.1
Region 6	2026-09-28 13:00	749.1	504.4	244.7	731.1	18.0
Region 7	2026-09-25 13:00	751.0	736.6	14.4	743.3	7.7
Region 8	2026-09-25 13:00	1016.6	1010.5	6.0	1011.8	4.8
Region 9	2026-09-23 13:00	809.0	796.9	12.1	804.0	5.0
Region 10	2026-09-28 13:00	743.0	502.3	240.7	734.8	8.3

Mean absolute error at the ten regional peak hours: baseline 159.9 kWh, model 7.3 kWh (all values in kWh).

Does the model improve on the baseline? Yes. Over all 1,680 test observations MAE falls from 42.71 to 4.55 kWh (−89.3 %) and RMSE from 79.40 to 5.79 kWh (−92.7 %), and it is better in all ten regions. Observation: the baseline's MAE is ordinary (about 4–5 kWh per day) on five of the

seven days and large only on Saturday 26 September (135.7 kWh) and Monday 28 September (139.5 kWh), the days after a weekday/weekend change, where yesterday is a poor guide; the model uses the weekday indicator and the week-ago value and stays near 4–5 kWh every day. This is our explanation of the cause, supported by the daily MAE values in results/forecast.json.

Is it useful for planning? The model's average error of 4.6 kWh is about 0.8 % of the mean hourly regional consumption (590.8 kWh), and its error at the regional peak hours is also small (2–18 kWh against peaks of 660–1,017 kWh), so it is accurate enough for day-ahead regional planning of supply and maintenance. Limits: the test week is a single week of one generator with a regular weekly pattern, so no holiday, weather or outage effects are tested; the totals use readings that were missing for about 1 % of planned readings, so the actual totals are slightly understated and the forecast is trained to reproduce this understatement; and in live use readings arrive up to three hours late, so the most recent hours of a lag source (and the latest day-end total) are incomplete at 00:00 and would need provisional values or a longer lag (such as lag_48h). Both the baseline and the model should therefore be monitored against late-corrected actuals.

Document contributions

Member	Student ID	Individual contributions
Mushambo Godfrey	101376	Overall project and System layout
Umuhoza Scovia	101374	Overall project and Graphical interpretation
Manzi Shema Christian	101382	Overporject and Doument preparation